WNDRVR
SUPPORT NETWORK

VXWORKS 7 VXSIM USER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 VxSim User's Guide, SR0650

2 September 2020

1. OVERVIEW

1. Introduction

The VxWorks® simulator (VxSim) is a simulated hardware target for use as a prototyping and test-bed environment for VxWorks. The VxSim allows you to develop, run, and test VxWorks applications on your host system.

Users link in applications and rebuild the VxWorks image exactly as they do in any VxWorks environment. The VxWorks image is executed on the host machine itself as a host process. There is no emulation of instructions, the code is based on the host architecture. A communication mechanism is provided to allow the simulator to obtain an Internet IP address and communicate with the Wind River tools on the host or with other nodes on the network using the VxWorks networking tools.

Because target hardware interaction is not possible, device driver development may not be suitable for simulation. However, the VxWorks scheduler is implemented in the host process, maintaining true tasking interaction with respect to priorities and preemption. This means that any application that is written in a portable style and with minimal hardware interaction should be portable between the VxSim and VxWorks.

The VxSim supports most VxWorks features available on target hardware and also provides support for a number of simulated hardware devices. The VxSim supports Linux and Windows operating systems.

2. VxWorks Simulator Overview

The VxWorks simulator utility, vxsim, supports a set of command-line configuration options to load a VxWorks simulator image, specify the boot parameters, configure memory options, and configure networking options for a simulated network.

VxSim supports three options for a simulation environment. One of the options is when no networking is required and is provided through the **passDev** boot device. The other two networking options for the VxWorks simulator are the network port redirection (also known as network address translation, or NAT), which is provided through the **simnet_nat** boot device; and full network simulation, which is provided through the **simnet** boot device.

The simulator supports three VxWorks boot devices. Unlike a real target boot device, the simulator boot device is not used to load VxWorks images since the image file is directly accessible.

Networking with the VxWorks Simulator

Two types of networking are available for the VxSim:

- Full network simulation provided with the simnet device
- Network port redirection provided with the simnet_nat device

simnet allows complex network simulation while simnet_nat provides a simple solution that is limited to UDP/TCP protocols. The main differences between these two networking methods are summarized in the following table.

Networking Method	Description	Limitations
simnet	Allows various protocol and subnet configurations.	You must start vxsimnetd before starting a simulator instance.

WNDRVR

Networking Method	Description	Limitations
	Uses the vxsimnetd daemon.	
simnet_nat	Allows access to network resources, including the Internet. Does not require the installation of any driver on the host. Does not require the use of vxsimnetd. Does not require specification of a gateway or target IP address.	Acts as a firewall by default, and as such does not permit any incoming traffic. Does not support protocols other than TCP and UDP. Therefore, it does not support ping and other ICMP utilities.

For more information on the simnet_nat device, see Network Redirection Using simnet_nat on page 19.

For an example usage of the simnet device in a simulated network environment, see Full Network Simulation Using simnet on page 27, Simulated Networks Across Multiple Hosts on page 59, and the networking examples in *VxWorks 7 Network Stack Programmer's Guide*.

Working with Pass-through Devices (passDev)

You use the passDev option when you do not require network simulation (simnet) or network redirection (simnet_nat). For passDev, no network device is created, no initialization is required, and you cannot connect to Workbench.

For example usage of a passDev device as a router, see the basic networking example in the *VxWorks 7 Network Stack Programmer's Guide*.

3. System Requirements

The minimum host system requirements for the VxWorks simulator are similar to those of a standard VxWorks installation.

For full details on host system requirements, see the *VxWorks 7 Release Notes*.

Host Operating System Support

The VxWorks 7 simulator supports a number of Windows and Linux operating systems. For full details on the supported versions of the operating systems, see *VxWorks 7 Release Notes*.

Symmetric Multiprocessing (SMP) Support

The host operating system (OS) simulates each CPU with a process that is scheduled by the host OS. Full SMP performance can be achieved when the host has more CPUs than the simulated target. Other scenarios with fewer CPUs are possible and may result in reduced performance.

> Note: If your host system supports hyper-threading, this can also be used to improve SMP simulation. For information on hyper-threading, see the architecture specific guides.

To set the number of CPUs for the simulator, configure the ENABLE_ALL_CPUS and VX_SMP_NUM_CPUS parameters of the **INCLUDE_KERNEL** component.

WNDRVR

Once the image is built, you can set the number of CPUs from 1 to **VX_SMP_NUM_CPUS** for a given simulation with the vxsim -ncpu command (see VxWorks Simulator Options Reference on page 14).

> Note: If you are using the VxWorks simulator to simulate multiple processors in SMP mode, you should be cautious when calling host functions from the VxWorks simulator. Review code that uses vxsimHostProcAddrGet( ) to make sure it is SMP-safe. Note that vxsimHostProcCall( ) replaces a direct function pointer dereference that is forbidden in SMP. For more information see Accessing Host OS Functions on page 85.

For more information on SMP, including configuration options, see the *VxWorks 7 Application Programmer's Guide: VxWorks SMP*.

64-Bit Support

To simulate 64-bit VxWorks, you must run the VxWorks simulator on a 64-bit host.

To set the data model for your VSB build, use the LP64 VSB option (see VSB Configuration Reference on page 4.

For information on 32-bit and 64-bit mode memory configurations, see Memory Configuration on page 76.

Compiler Support

The VxWorks simulator image is built with the Clang compiler (no other compilers are supported for this purpose). For information on compiler options see VxWorks Simulator Options Reference on page 14 and Compiler Options on page 69.

Simulated Hardware Support

To support application development, the VxWorks simulator provides simulated hardware support for the following hardware-related features:

- Non-Volatile RAM Support
- Pass-Through File System (passFS)
- Serial Host Line Support
- Serial Input and Output (SIO)
- Timers
- Timestamp Driver
- Virtual Disk Support

For information on these simulated hardware features for your VxWorks simulator image, see Configuring Hardware Simulation on page 81.

Limitations

The VxWorks simulator is not suitable for all prototyping needs.

The VxWorks simulator executes on the host machine and does not attempt the simulation of machine-level instructions for a target architecture. For this reason, the VxWorks simulator is not a suitable basis on which to develop hardware device drivers. However, the VxWorks simulator includes MMU support and implements the architecture-specific part of a memory management unit to provide the same level of feature support as hardware-based targets.

WNDRVR

2. CONFIGURATION AND BUILD

1. Configuration and Build Workflow

Before beginning any VxWorks development without using pre-built kernel libraries or a pre-built image, you must build a VSB and a VIP.

The pre-built 32-bit mode kernel libraries are based on the default VSB options (see VSB Configuration Reference on page 4) and the pre-built kernel image is based on the default VIP components (see Kernel Configuration Components (VIP) on page 5). You can run and debug your applications with the prebuilt images, doing such tasks as building a DKM, building an RTP, or connecting the debugger.

The pre-built VxWorks Source Build (VSB) project and the pre-built VxWorks Image Project (VIP) located in *installDir* **/vxworks-7/samples/prebuilt_projects/**. Before beginning any VxWorks development without using pre-built kernel libraries or a pre-built image, you must build a VSB and a VIP. For specific instructions on how to configure and build a VSB and a VIP, and how to create a downloadable kernel module (DKM) or a real-time process (RTP) application, see the *VxWorks 7 Configuration and Build Guide*.

For a list of available source build configuration options for the VxWorks simulator, see VSB Configuration Reference on page 4.

For a list of available kernel configuration components for the VxWorks simulator, see Kernel Configuration Components (VIP) on page 5.

2. BSP and CPU Support

The VxWorks simulator supports BSPs and CPUs for Windows and Linux host environments.

The following table lists the available BSPs and CPUs for the VxWorks simulator.

Host Type	BSP Name	CPU Name
Windows	vxsim_windows	**SIMNT**
Linux	vxsim_linux	**SIMLINUX**

3. VSB Configuration Reference

VxSim VxWorks Source Build (VSB) configuration provides alternatives to the default configuration for the simulator.

The following table lists the relevant available VSB options for the VxSim.

Name	Description	Default
VXSIM	Enable this option to provide support for the VxWorks simulator architecture.	Enabled
FP_SIMNT_hard_32	Enable this option to support hardware-based floating point operations, for 32-bit mode.	Enabled
ENDIAN_SIMNT_little_32	Enable this option to select the little-endian storage method for bytes, for 32-bit mode. This option must be enabled for Windows and Linux environments.	Enabled
FBDEV_VXSIMFB	Enable this option to include the simulator frame buffer driver.	Disabled
BDM	Enable block device managers (BDMs) for various flash storage devices.	Enabled
LP64	Enable this option to support the 64-bit mode in VxWorks (LP64 data model). When this option is not selected the 32-bit mode is supported (ILP32 data model). For more information on 32-bit and 64-bit modes, see *Porting C code from 32-Bit to 64-Bit* application note.	Disabled
SMP	Enable this option to support Symmetric Multiprocessing (SMP) in the VxWorks kernel. For more information on SMP, see *Symmetric Multiprocessing (SMP) Support* in System Requirements on page 2.	Enabled
STANDARD_STL	Enable this option to support the standard libraries.	Enabled
ABRIDGED_STL	Enable this option to support the abridged version of the standard libraries.	Disabled
DEBUG_FLAG	Enable this option to allow the building of all source code with the -g flag set (debug flag). Optimization compiler flags such as -O3 are disabled by default.	Disabled

4. Kernel Configuration Components (VIP)

The VxSim VxWorks Image Project (VIP) components enable the configuration of the kernel image for your operating environment.

The following table lists the VIP components for the VxSim.

Name	Description	Default
DRV_INT_VXSIM	VxBus interrupt controller (IntCtrl) for the simulator.	Included
DRV_HOST_SIO_VXSIM	VxBus driver for the VxWorks simulator host serial input and output (SIO).	Excluded
DRV_NET_VXSIM	VxWorks simulator enhanced network driver (END).	Included
DRV_SIO_VXSIM	VxBus driver for the VxWorks simulator serial input and output (SIO).	Included
DRV_TIMER_VXSIM	VxBus driver for the VxWorks simulator timer.	Included
INCLUDE_VXSIM_USER_SM	Include the support for user reserved shared memory.	Excluded
VXSIM_USER_SM_SIZE	Defines the size of the VxWorks simulator user shared memory. Default is 8 kbytes.	0x2000
INCLUDE_ADR_SPACE_LIB	Include support for the system address space allocator. See the diagrams *32-Bit VxWorks System Memory Layout (VxWorks Simulator)* and *64-Bit VxWorks System Memory Layout (VxWorks Simulator)* in Memory Configuration on page 76 for system memory maps. The mode (32-bit or 64-bit) is set with the LP64 VSB option.	Included
VXSIM_KERNEL_VIRT_POOL_RGN_SIZE	Defines the size of the kernel virtual pool. Default is 256 Mbytes for the 32-bit mode and 4 Gbytes for the 64-bit mode.	0x1000 0000 (32-bit) 0x1 0000 0000 (64-bit)
VXSIM_RTP_PRIVATE_RGN_SIZE	Defines the size of the RTP user virtual memory region. Default is 256 Mbytes for the 32-bit mode and 4 Gbytes for the 64-bit mode.	0x1000 0000 (32-bit) 0x1 0000 0000 (64-bit)
VXSIM_SHARED_RGN_SIZE	Defines the size of the shared user virtual memory region. Default is 256 Mbytes for the 32-bit mode and 4 Gbytes for the 64-bit mode.	0x1000 0000 (32-bit) 0x1 0000 0000 (64-bit)
VXSIM_SIMNET_MEM_RGN_SIZE	Defines the size of the simnet memory pool. Default is 16 Mbytes for 32-bit and 64-bit modes.	0x100 0000

WNDRVR

Additional VIP Components

To create a VIP compatible with all examples provided for the VxSim, you must include additional components in your VIP.

Include the following components in your VIP build:

- Add INCLUDE_IPPING_CMD to access the ping shell command and test network connectivity in an IPv4 environment.
- Add INCLUDE_ROUTECMD to access the route shell command and enable the configuration of routing tables.

3. WORKING WITH VXWORKS SIMULATOR

1. Launching the VxWorks Simulator from Workbench

By default, Workbench starts the VxWorks simulator with a simnet_nat device that does not require the VxWorks simulator network daemon (vxsimnetd). To use a simnet device, you must start vxsimnetd before launching the simulator instances.

Getting Help

To get help for the vxsimnetd command, enter vxsimnetd -help at the host command shell prompt.

To get help for the vxsimnetd shell, enter help at the vxsimnetd shell prompt.

To get help for the vxsim command, open a development shell from Workbench and enter vxsim -help at the prompt.

For more information on vxsimnetd, see VxWorks Simulator Network Daemon Configuration and Use on page 40.

For configuration details, see VxWorks Simulator Options Reference on page 14.

For information on vxsim commands available from Workbench, see Configuring vxsim from Workbench on page 17.

For information on port redirection, see Network Redirection Using simnet_nat on page 19.

For information on network simulation, see Full Network Simulation Using simnet on page 27.

For information on simulator memory configuration, see Memory Configuration on page 76.

About This Task

To launch the VxWorks simulator from Workbench and connect to a target, proceed as follows:

Procedure

1. Start the VxWorks simulator network daemon:
   1. Open a Windows command shell as an administrator.

      Note: If you are performing this procedure on Linux, open a command shell and log in with root privileges.

   2. From the command shell, access the *installDir* **\vxworks-7\host\x86-win32\bin** directory and use one of the following methods to launch the simulator:

      ```
      > vxsimnetd
      ```

      This command starts the daemon with the default network configuration.

      ```
      > vxsimnetd -f configurationFile.conf
      ```

      This command syntax starts the daemon with a custom network configuration.

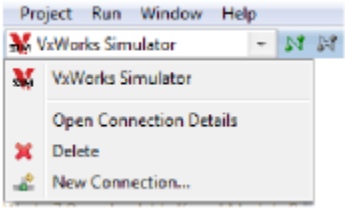
      ```
      > vxsimnetd -f configurationFile.conf -s
      ```


WNDRVR

This command syntax starts the daemon with a custom network configuration and launches the vxsimnetd shell.

The vxsimnetd network daemon launches.

2. From the **Workbench** toolbar, select **New Connection**:



3. Enter a connection name.

   The default connection name is **VxWorks Simulator**.

4. From the Target panel, set the **Target Type** field to VxWorks Simulator.

   The Connection Mode is set to Application Mode.

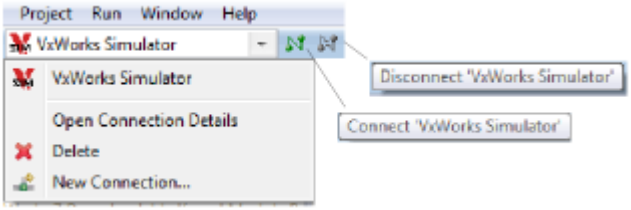
5. From the Configuration panel, set the **Kernel Image** field to point to the kernel image file (vxWorks executable).
6. From the Configuration panel, click **Advanced**.
7. From the Advanced dialog, set the **Network Config** field as required (simnet or NAT (simnet_nat)).

   If you select NAT, you can click **Configure** and set the port mappings in the Configure Network dialog, then click **OK**.

   If you select simnetd, specify the simulator IP address and TCP port number. The default is 192.168.200.1 for the IP address and 1534 for the port number.

8. In the **Additional Options** field, you can add vxsim configuration options as follows:
   - All boot parameter options except -p, -f, -d, and -e
   - Memory option: - size
   - Network option: -ni
   - Miscellaneous options: -nvram and -tmpdir
9. Click **OK** to close the Advanced dialog.
10. Click **Apply** to save the configuration.

    From the Workbench toolbar, observe that the VxWorks Simulator connection is displayed.



11. Click **Connect** to launch the simulator and connect to the target.

    Observe the VxWorks simulator memory configuration and the presence of the kernel shell prompt on the VxWorks Simulator tab within the Terminals tab. For example:

```
Checking License...OK
Virtual Base Address: 0x10000000
```


WNDRVR

```
Virtual Top Address: 0x48000000 Virtual Size: 0x38000000 (896Mb)
Physical Base Address: 0x10000000
Physical Top Address: 0x18000000 Physical Size: 0x08000000 (128Mb)
Loading...
...
```

Results

To reboot the instance, enter reboot at the kernel shell prompt.

To terminate the connection, click the **Disconnect** button on the tool bar.

To reconnect, click the **Connect** button on the toolbar.

To terminate a Windows simulation, click the **Disconnect** button on the tool bar and close the VxWorks Simulator console window.

To terminate a Linux simulation, click the **Disconnect** button on the tool bar and enter Ctrl+\ in the simulator console.

To change the configuration of the connection, select **Open Connection Details**.

To re-launch the simulator instance and reconnect, click the **Connect** button.

2. VxSim Controls Reference

You can take actions such as stopping and restarting a VxSim session.

Task	Workflow
Reboot the instance	Enter Ctrl+X or reboot in the simulator console window.
Terminate the connection	Click the **Disconnect** icon on the tool bar.
Reconnect	Click the **Connect** icon on the toolbar.
Terminate a Windows simulation	Click the **Disconnect** icon on the tool bar and close the VxWorks Simulator console window.
Terminate a Linux simulation	Click the **Disconnect** icon on the tool bar and type Ctrl+\ in the simulator console.
Note: Issuing the exit command in the kernel shell terminates the shell session only and cannot be used to exit the VxWorks simulator instance. To programmatically exit the instance (which may be useful for scripting), you can reboot (BOOT_NO_AUTOBOOT). If you wish to use this option, you must start the VxWorks simulator instance with the -exitOnError option.	
Change the configuration of the connection	Select Open Connection Details.
Relaunch the simulator instance and reconnect	Click the **Connect** icon.

3. Using Multiple Simulator Instances

You can use either simnet or simnet_nat to launch multiple simulator instances using the same VxSim image.

Getting Help

To get help for the vxsim command, use the vxsim -help command from the development shell.

To get help for the ping command, type ping at the command prompt.

For more information on vxsim, see VxWorks Simulator Options Reference on page 14.

For information on network simulation, see Full Network Simulation Using simnet on page 27.

For information on port redirection, see Network Redirection Using simnet_nat on page 19.

Using simnet to Launch Multiple Simulator Instances on page 11 Use simnet to launch multiple simulator instances using the same VxSim image.
Using simnet_nat to Launch Multiple Simulator Instances on page 12 Use simnet_nat to launch multiple simulator instances using the same VxSim image.

3.1. Using simnet to Launch Multiple Simulator Instances

Use simnet to launch multiple simulator instances using the same VxSim image.

Before You Begin

You must open a VxWorks development shell for your host.

- For Windows:

```
wrenv -p vxworks-7
```

- For Linux:

```
./wrenv.sh -p vxworks-7:
```

Procedure

1. From the development shell, access the default directory of the VIP and execute the following commands to configure and launch the instances:

```
>cd installDir /workspace /vipDir /default
>vxsim -d simnet -e 192.168.200.1
>vxsim -d simnet -e 192.168.200.2 -p 1
```

   Each vxsim command starts a VxSim instance that attaches to subnet 192.168.200.0. The second vxsim command adds the -p parameter to assign a value of 1 as the processor number to the second simulator instance (VxSim1). If you do not specify a -p number, the default value of 0 is applied (VxSim0). Simulator instances running concurrently must have a unique processor number.

2. From the VxSim0 window, ping the VxSim1 instance at 192.168.200.2 and observe the results (0% packet loss).

```
>cmd
[vxWorks *]# ping 192.168.200.2
Pinging 192.168.200.2 (192.168.200.2) with 64 bytes of data:
Reply from 192.168.200.2 bytes=64 ttl=64 seq=0 time<1ms
--- 192.168.200.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0 ms
rtt min/avg/max = 0/0/0 ms
[vxWorks *]#
```

From the VxSim1 window, ping the VxSim0 instance at 192.168.200.1 and observe the results.

```
>cmd
[vxWorks *]# ping 192.168.200.1
...
```

Parent topic: Using Multiple Simulator Instances on page 11

3.2. Using simnet_nat to Launch Multiple Simulator Instances

Use simnet_nat to launch multiple simulator instances using the same VxSim image.

Before You Begin

You must open a VxWorks development shell for your host.

- For Windows:

  ```
  wrenv -p vxworks-7
  ```

- For Linux:

  ```
  ./wrenv.sh -p vxworks-7:
  ```

Procedure

From the development shell, access the default directory of the VIP and execute the following commands to configure and launch two or more simnet_nat simulator instances using the same VxSim image. For example:

```
>cd installDir /workspace /vipDir /default
>vxsim -d simnet_nat
>vxsim -d simnet_nat -p 1
```

Parent topic: Using Multiple Simulator Instances on page 11

4. Using a Telnet Client to Display the Simulator Console

Access and display the VxSim console from a telnet client.

Procedure

1. Use the -add_dev command option to launch an instance of the VxSim and a Telnet server for displaying its console. The syntax of the option is:

   ```
   vxsim -add_dev serial0=telnet:[portNumber]:[nowait],[nokill]
   ```

   where the optional parameters are interpreted as follows:

   portNumber

   The port to use; if omitted, the VxSim chooses the port number.

   nowait

   The VxSim starts without waiting for the client to connect.

   nokill

   Allows the console to be closed and restarted without killing the VxSim process.

   Examples:

   ```
   -> vxsim -add_dev serial0=telnet
   ```

   ```
   -> vxsim -add_dev serial0=telnet:5656:nowait
   ```

   ```
   -> vxsim -add_dev serial0=telnet::nowait
   ```

   Note the double colon (::) in the last example. It is required when no port number is specified and causes the VxSim to supply a port number.

2. If you did not specify a port number for the Telnet server with the -add_dev option, use the -list_dev option of the vxsim command to obtain the port number.

   The -list_dev option has the following forms:

   ```
   -> vxsim -list_dev serial0
   -> vxsim -list_dev all
   ```

   Both forms of the option are equivalent, because only a serial device used for displaying the console over Telnet is supported.

   On Linux, the -list dev option displays the port number on the standard output.

   On Windows, the VxSim is not a console application, and VxSim output is not displayed on the standard output. To obtain the port number, you need to use -list_dev with the -l option. This writes the port number to the log file specified with -l.

   Output from -list dev appears as follows:

   ```
   serial0=telnet:portNumber
   ```

3. Connect a Telnet client to the Telnet server:

   ```
   > telnet localhost portNumber
   ```


WNDRVR

Results

The preceding steps display the VxSim console in a new window. You can also display the console directly in Workbench.

5. Disconnecting from the VxWorks Simulator

The behavior when you disconnect from the console depends on the options you specified when starting VxSim.

About This Task

If you start the VxSim with the following command, the simulator terminates when you terminate the telnet session.

```
vxsim -add_dev serial0=telnet:5656
```

If you start the VxSim with the following command, the VxSim remains active when you terminate the telnet session.

```
vxsim -add_dev serial0=telnet:5656:nokill
```

Procedure

To terminate the simulation, use the following commands:

- On Windows:

  ```
  -> reboot(1)
  ```

- On Linux:
  CTRL+\ or

  ```
  % reboot(1)
  ```

6. VxWorks Simulator Options Reference

The VxSim utility vxsim supports a set command-line options to load a VxSim image, specify the boot parameters, configure memory options, and configure networking options for a simulated network.

The simulator instance can be launched as a pass-through device (passDev - default), a simulated networked device (simnet), or as a network address translator (NAT) device (simnet_nat).

From a development shell, use vxsim -help to see a complete list of options.

The following table lists the vxsim options available from the command-line.

VxWorks Simulator Option	Description
-add_dev serial0= telnet:[*portNumber*]: [nowait],[nokill]	Launches an instance of the simulator that makes its console available over a Telnet server. For a discussion of its usage and an explanation of

VxWorks Simulator Option	Description
	its optional parameters, see Disconnecting from the VxWorks Simulator on page 14.
-add_nat_redir *networkRedirection* -anr	Add a NAT network redirection. For details on the syntax and usage of this option, see Adding Network Redirections on page 23.
-backplane *inetOnBackplane* -b *inetOnBackplane*	The backplane address of the target system.
-device *bootDevice* -d *bootDevice*	The type of device to boot from: passDev, simnet, or simnet_nat. The default is passDev.
-ethernet *inetOnEthernet* -e *inetOnEthernet*	The Internet address to assign to the target system (the boot interface).
-exitOnError	Upon error, exit the VxSim instance without prompting for user input.
-file *fileName* -f *fileName*	The file containing the VxWorks image to load. If no file is specified, the vxWorks file, if any, in the current directory is loaded.
-flags *flags*	Configuration flags. The default is 0.
-gateway *gatewayInet* -g*gatewayInet*	The Internet address of the gateway.
-help	Print a help message listing the command-line options.
-hostinet *hostinet* -h *hostinet*	The host Internet address.
-hostname *hostName* -hn *hostName*	The host machine to boot from. The default value for this option on a Windows system is **host**. On a Linux system, the default value is always the actual host name.
-kill *processNumber* -k *processNumber*	Kills the VxSim instance referenced by *processNumber*.
-list_dev serial0 -list_dev all	In the current release, this option provides the port number used by the Telnet server for the current instance of the VxSim (see the table entry for the -add_dev option). Currently, both forms of the option are equivalent, because only a serial device used for displaying the console over Telnet is supported. On Linux, the port number is displayed on the standard output. On Windows, the VxSim is not a console application, and VxSim output is not displayed on the standard output. To obtain the port number, you need to use -list_dev with the -l option. This writes the port number to the log file specified with -l. Examples:

VxWorks Simulator Option	Description

	`% vxsim...-list_dev serial0`
	`% vxsim...-l logfile.txt -list_dev serial0`
	`% vxsim...-l list_dev all -p 3 -l logfile.txt -lc`
	The first example, with no -l option, only applies to Linux.
	For more information on the usage of the -list_dev option, see Using a Telnet Client to Display the Simulator Console on page 12.

-list_nat_redir -lnr	List all NAT network redirections.
-logclean -lc	Clears the log file specified by the -l option. This allows you to start an instance of the VxSim, clear the log file used by a previous instance, and write new data to it. When the VxSim reboots, the log file is also cleared.
-logfile *logFile* -l *logFile*	Enables output logging.
-ncpu *n*	Limits the number of CPUs to *n*. Allows a VxSim image configured with n CPUs to run with 1 to *n* CPUs. The maximum number of CPUs is defined at build time by the VSB parameter **VX_SMP_NUM_CPUS**.
-netif *additionalInterface* -ni *additionalInterface*	VxSim additional network interfaces. For example usage, see Using Multiple Network Interfaces on page 33.
-nice	When you include the -nice option in your boot line sequence, it reduces the priority of the VxSim process. On Windows, the -nice option lowers the priority of the vxsim process by one priority class (for example, at **BELOW_NORMAL** rather than **NORMAL**). On Linux, the -nice option reduces the priority of the vxsim process by 10
-nvram *pathToFileForNvram* -n *pathToFileForNvram*	The VxSim non-volatile RAM file.
-other *other* -o *other*	Unused, available for applications.
-password *ftpPassword* -pw *ftpPassword*	The user's password (for FTP only).
-processor *number* -p *number*	Sets the processor number, which is effectively an identifier for this VxSim instance. The default value is 0.

WNDRVR

VxWorks Simulator Option	Description
-prot-level *protectionLevel* -pl *protectionLevel*	Sets the VxSim memory protection level. Possible values include min (0), max (1), or an integer. The default value is max (1).
-size *memorySize* -memorySize *memorySize*	The VxSim memory size in bytes. The default is 32 MB.
-startup *script* -s *script*	A startup script for the kernel shell.
-targetname *targetName* -tn *targetName*	The name of the target. The default value is **vxsim** for the vxsim0 instance; vxsim*CpuNum* for all other instances.
-tmpdir*directory*	The directory for temporary VxSim files. The default on a Windows system is the **TEMP** environment variable value; on a Linux system, the default is **/tmp**.
-unit *unitNumber*	The network device unit number (for the boot interface). The default is 0.
-username *userName* -user *userName*	The user name used to access the host.
-version -v	Print the simulator version.

Note: When issuing these command options using the vxsim command from the command line on a Linux host, use double quotation marks around the option values. For example, use vxsim -ni "simnet" in place of vxsim -ni simnet.

Configuring vxsim from Workbench on page 17 Add vxsim configuration options from the **Additional Options** field of Workbench.

6.1. Configuring vxsim from Workbench

Add vxsim configuration options from the **Additional Options** field of Workbench.

About This Task

For information on the available options, see VxWorks Simulator Options Reference on page 14.

Procedure

1. Set boot parameter options.

   Set all boot parameter options except -p, -f, -d, and -e.

2. Set the memory -size option.
3. Set the network -ni option.
4. Set the miscellaneous -nvram and -tmpdir options.

WNDRVR

Parent topic: VxWorks Simulator Options Reference on page 14

4. WORKING WITH NETWORK REDIRECTION

1. Network Redirection Using simnet_nat

The simnet_nat device provides network redirection, also known as network address translation (NAT).

The operational aspects of simnet_nat are as follows:

- allows access to network resources, including the Internet
- acts as a firewall by default and, as such, does not permit any incoming traffic
- only supports TCP and UDP; therefore, does not support ping and other ICMP utilities
- does not require the installation of any driver on the host
- does not require the use of the VxSim network daemon, vxsimnetd
- does not require specification of a gateway or target IP address

2. NAT Limitations

The NAT capability for the VxSim has a number of limitations.

The network redirection limitations are as follows:

- Although you can start several simulator instances using the simnet_nat device on the same host, doing so requires that each port redirection be different; that is, multiple simulator instances cannot use the same host port.
- The network redirection facility supports IPv4 and TCP/UDP packets only. It does not support IPv6.
- You can configure the VxSim to include both simnet_nat and simnet devices, or you can configure it with a simnet_nat device only. In either case, however, to use network redirection, the simnet_nat device must be your boot device. You cannot add a simnet_nat device through the -ni command-line option.
- The simnet_nat device cannot be manually loaded from VxWorks.
- You cannot change the gateway IP address and simnet_nat network device IP address from VxWorks.
- Multiple instances of the VxSim that are configured to include network redirection cannot communicate using the network-redirection IP addresses (10.0.10.x). However, a single redirection-enabled simulator instance can access an IP port on another redirection-enabled simulator instance by means of the host IP address and the host redirected port.

3. Simulating a NAT Device Using simnet_nat

The VxSim provides a user-mode network stack that acts as a bridge between the guest network stack and the host network stack without the need for a specific network driver.

About This Task

This type of connection allows access to network resources, including the Internet, and does not require a host connection driver (WRTAP driver) or the VxWorks network daemon, vxsimnetd. By default, it acts as a firewall and does not permit any incoming traffic. It also does not support protocols other than TCP and UDP; therefore, ping and other ICMP utilities do not work with this type of connection.

WNDRVR

Getting Help

To get help for the wrenv command, type wrenv -help at the development shell prompt.

To get help for the vxsim command, type vxsim -help at the development shell prompt.

To get help for the kernel shell (->), type help at the kernel shell prompt.

For more information on vxsim, see VxWorks Simulator Options Reference on page 14.

For information on memory configuration, see Memory Configuration on page 76.

For information on port redirection, see Adding Network Redirections on page 23.

To start a VxSim instance using a simulator-based VIP image with default redirection, proceed as follows:

Procedure

1. Create a simulator-based VSB and VIP.

   See Configuration and Build Workflow on page 4 for more information.

2. Open a development shell either from Workbench 4 or from the command line.

   From the command line, use the following command:

```
installDir> wrenv -p vxworks-7
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
installDir>
```

3. Access the default directory of the VIP.

   For example:

   installDir **/workbench-4/workspace/my_vip/default**

4. Enter the following command to launch the simulator instance with a simnet_nat boot device (-d) for network address translation (NAT):

```
>vxsim -d simnet_nat
Checking License...OK
Virtual Base Address: 0x10000000
Virtual Top Address: 0x48000000 Virtual Size: 0x38000000 (896Mb)
Physical Base Address: 0x10000000
Physical Top Address: 0x18000000 Physical Size: 0x08000000 (128Mb)
Loading...
0x3780f0 + 0x3d9f0
Redirecting TCP port 50642 to 10.0.10.2:21
Redirecting TCP port 50643 to 10.0.10.2:22
Redirecting TCP port 50644 to 10.0.10.2:23
Redirecting TCP port 50645 to 10.0.10.2:69
Redirecting TCP port 50646 to 10.0.10.2:80
Redirecting TCP port 50647 to 10.0.10.2:443
Redirecting TCP port 50648 to 10.0.10.2:1534
```


WNDRVR

```
...
->
```

Note the default port redirections. The default port redirections are stored in *installDir* **/vxworks-7/host/resource/vxsim/vxsim_nat_redirs.txt**.

To access a guest UDP/TCP port, the port must be explicitly redirected.

To configure port redirection, use the vxsim -add_nat_redir or -anr commands.

To view the current port redirections, use the vxsim -list_nat_redir or -lnr commands.

Viewing the Network and Boot Configurations on page 21 View the network configuration for the target.

3.1. Viewing the Network and Boot Configurations

View the network configuration for the target.

About This Task

Use the show bootline command from the command shell to view the current boot parameters. To modify the boot parameters use the set bootline or bootChange command.

For more information on boot parameters and how to configure boot parameters, see Boot Parameters on page 80.

Getting Help

To get help for the ifconfig command, type ifconfig "" at the kernel shell prompt.

Procedure

Use the ifconfig command from the kernel shell. For example:

```
-> ifconfig
ifconfig
...
simnet_nat0Link type:Ethernet HWaddr 7a:7a:0a:00:00:02
inet 10.0.10.2 mask 255.255.255.252 broadcast 10.0.10.3
inet6 unicast fe80::787a:aff:fe00:2%simnet_nat0 prefixlen 64 automatic
UP RUNNING SIMPLEX BROADCAST MULTICAST
MTU:1500 metric:1 VR:0 ifindex:2
RX packets:76 mcast:0 errors:0 dropped:0
TX packets:85 mcast:8 errors:0
collisions:0 unsupported proto:0
RX bytes:24k TX bytes:23k
value = 0 = 0x0
->
```

The target IP address is 10.0.10.2.

Parent topic: Simulating a NAT Device Using simnet_nat on page 19

WNDRVR

4. Using Network Redirection with simnet_nat

The VxWorks simulator provides network redirection through the simnet_nat device. On any given VxSim instance, only one simnet_nat device can be configured.

The simulator stack is responsible for the following process:

1. Intercept all network packets from the VxWorks guest.
2. Analyze the network packets.
3. Filter out non-UDP and TCP packets.
4. Forward the UDP/TCP packets to the correct destination by creating a UDP/TCP socket on the host.

The simulator stack also performs subsequent tasks:

1. Monitor the activity on the newly created UDP/TCP sockets.
2. Fix the headers on any received UDP/TCP packets.
3. Forward those packets to the simulator network driver.

No particular setup is necessary to access the network from the simulator, and port redirections are automatic. When you start the simulator and specify simnet_nat as the boot device, a number of ports are automatically redirected. Redirections are printed to the console. For example:

```
Redirecting TCP port 58811 to 10.0.10.2:21
Redirecting TCP port 58812 to 10.0.10.2:22
Redirecting TCP port 58813 to 10.0.10.2:23
Redirecting TCP port 58814 to 10.0.10.2:69
Redirecting TCP port 58815 to 10.0.10.2:80
Redirecting TCP port 58816 to 10.0.10.2:443
Redirecting TCP port 58817 to 10.0.10.2:17476
Redirecting UDP port 44071 to 10.0.10.2:17185
Redirecting TCP port 58818 to 10.0.10.2:1534
Redirecting TCP port 58819 to 10.0.10.2:5698
Redirecting TCP port 58820 to 10.0.10.2:5678
Redirecting TCP port 58821 to 10.0.10.2:3333
Redirecting TCP port 58822 to 10.0.10.2:49152
Redirecting TCP port 58823 to 10.0.10.2:49153
Redirecting TCP port 58824 to 10.0.10.2:49154
Redirecting TCP port 58825 to 10.0.10.2:49155
Redirecting TCP port 58826 to 10.0.10.2:49156
Redirecting TCP port 58827 to 10.0.10.2:49280
Redirecting TCP port 58828 to 10.0.10.2:49281
Redirecting TCP port 58829 to 10.0.10.2:49282
Redirecting TCP port 58830 to 10.0.10.2:49283
Redirecting TCP port 58831 to 10.0.10.2:49284
```

In the console output above:

- target or guest (that is, VxWorks) TCP port 21 (FTP) can be reached through host TCP port 58811
- target TCP port 22 (SSH) can be reached through host TCP port 58812
- target TCP port 23 (telnet) can be reached through host TCP port 58813, and so on

If VxWorks is configured to provide an SSH or telnet server, you would use the following to reach the target SSH and telnet server:

```
ssh -p 58812 hostIp
telnet -p 58813 hostIp
```

Overriding Default Redirections

Default redirections do not specify the host port value, since it is not possible to obtain a value that will always be available. This means that the values of 58811, 58812, and 58813 shown above might change if the simulator is restarted.

However, you can overwrite the default configuration by specifying a new guest port configuration. To do so, use the vxsim command-line or use Workbench (see Adding Network Redirections on page 23). The settings override the default configuration.

The default port redirections are stored in *installDir* **/vxworks-7/host/resource/vxsim/vxsim_nat_redirs.txt**.

Adding Network Redirections on page 23 Besides the default redirections that are automatically performed, you can add a custom redirection, such as to connect a custom server application that is running on the simulator.
Listing Network Redirections on page 25 You can view the list of redirected ports for a particular simulator instance—for example, as part of a script.

4.1. Adding Network Redirections

Besides the default redirections that are automatically performed, you can add a custom redirection, such as to connect a custom server application that is running on the simulator.

You can add a custom redirection either from the command line or from within Workbench.

Adding Network Redirections from the Command Line on page 23 Add a custom network redirection from the command line.
Adding Network Redirections from Workbench on page 24 Add a custom network redirection from Workbench.

Parent topic: Using Network Redirection with simnet_nat on page 22

Adding Network Redirections from the Command Line

Add a custom network redirection from the command line.

Procedure

To add a network redirection, use the -add_nat_redir or -anr option. The syntax for this option is as follows:

```
vxsim -add_nat_redir udp|tcp:guestPortNumber [[:hostPortNumber [:portName]]
```

udp

Redirect a UDP port.

tcp

Redirect a TCP port.

guestPortNumber

The port number of the guest port that you want to access.

WNDRVR

hostPortNumber

The host port number that will be used to access the specified *guestPortNumber* (an optional parameter). If you do not define *hostPortNumber*, the simulator automatically allocates a port number.

portName

A string that Wind River Workbench uses to identify the port's purpose when the host port number may vary. You can use this string so that you don't have to remember the port number.

To specify multiple redirections, separate them with a semicolon character (;). For example:

To redirect guest UDP port 0x4321 to host UDP port 2000

```
udp:0x4321:2000
```

To redirect guest UDP port 0x4321 to a random host UDP port

```
udp:0x4321
```

To redirects guest UDP port 0x4321 and guest TCP port 23 to random host UDP and TCP ports

```
udp:0x4321;tcp:23
```

Parent topic: Adding Network Redirections on page 23

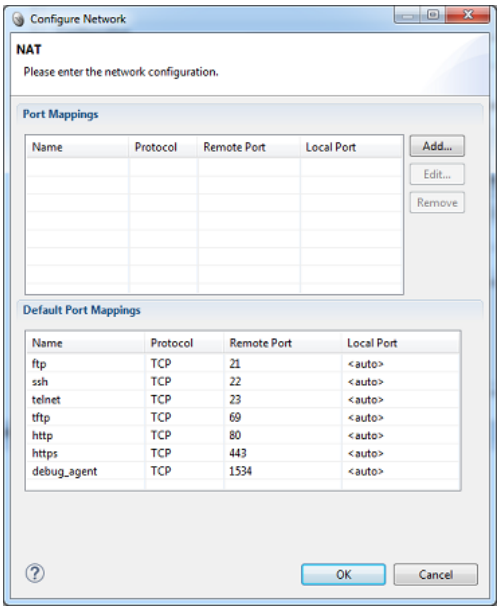
Adding Network Redirections from Workbench

Add a custom network redirection from Workbench.

About This Task

Procedure

Specify the port name, desired protocol, remote port (guest port), and local port (host port) for each entry in the Configure Network dialog box of Workbench (see Launching the VxWorks Simulator from Workbench on page 8) shown below.



Parent topic: Adding Network Redirections on page 23

4.2. Listing Network Redirections

You can view the list of redirected ports for a particular simulator instance—for example, as part of a script.

You can view the full list of redirections on Linux or Windows through the -list_nat_redir (or -lnr) command-line option to the simulator.

Listing Network Redirections for Linux Hosts on page 25 View the full list of redirections on Linux.
Listing Network Redirections for Windows Hosts on page 25 View the full list of redirections on Windows.

Parent topic: Using Network Redirection with simnet_nat on page 22

Listing Network Redirections for Linux Hosts

View the full list of redirections on Linux.

Procedure

To identify which simulator instance you want information about, use the -p option to specify the corresponding processor number. For example:

```
% vxsim -list_nat_redir -p 0
tcp:23:2000:telnet
```

Parent topic: Listing Network Redirections on page 25

Listing Network Redirections for Windows Hosts

View the full list of redirections on Windows.

About This Task

On Windows hosts, the results of -list_nat_redir are not printed to standard output.

Procedure

To view the output, use the -l option to save the VxSim output to a file, and then print the contents of the file. For example:

```
> vxsim -list_nat_redir -l %TEMP%\output.txt -lc -p 0
> cat %TEMP%\output.txt
tcp:23:2000:telnet
```

Parent topic: Listing Network Redirections on page 25

5. Accessing the VxWorks Simulator from a Remote Host

When you use network redirection through the simnet_nat device, the simulator instance can be reached through the host it is running on.

For example, if VxWorks telnet port 23 is redirected to host port 12345, and the host has the IP address 10.10.10.1, you would use the following sequence to connect on the simulator telnet server:

```
telnet 10.10.10.1 -p 12345
```


WNDRVR

5. WORKING WITH FULL NETWORK SIMULATION

1. Full Network Simulation Using simnet

The simulated network device (simnet) enables full network simulation.

The VxWorks simulator provides support for setting up a subnet, or multiple subnets using a network of VxWorks simulator instances.

The operational aspects of simnet are as follows:

- Allows various protocol and subnet configurations to simulate network devices.
- Uses host connection drivers that allow subnets to communicate with your host. See Host Connection Drivers—TAP/WRTAP on page 55.
- Uses the VxSim network daemon, vxsimnetd, to create and configure subnets. See VxWorks Simulator Network Daemon Configuration and Use on page 40.

To simulate a single class C subnet, proceed as follows:

1. Create a simulator-based VSB and VIP.
   See Configuration and Build Workflow on page 4.
   To test connectivity, add the ping shell command component (**INCLUDE_IPPING_CMD**) to your VIP. See the discussion of *Additional VIP Components* in Kernel Configuration Components (VIP) on page 5.
2. For a Windows environment, install a host connection driver (WRTAP driver) and configure network connectivity to enable IP communications between the VxSim and your host. See Installing a WRTAP Driver on page 62. For Linux, use the TAP driver. See the discussion of *Linux Hosts* in Installing the Host Connection Driver on Linux on page 57.
3. Configure and start the VxSim network daemon vxsimnetd. See Configuring and Starting the VxWorks Simulator Network Daemon on page 28.
4. Configure and start a VxSim instance with vxsim. See Launching a VxWorks Simulator Instance from the Command Line on page 29.
5. View the current network and boot configuration of the simulator instance. See Viewing the Network and Boot Configurations on page 31.
6. Test the connection between your host and the simulator instance. See Testing a Simulated Network on page 31.

Installing a WRTAP Driver on page 62 To simulate network IP connectivity to your host operating system, install a WRTAP driver.
Configuring and Starting the VxWorks Simulator Network Daemon on page 28 The VxSim provides a full network simulation environment through the VxSim network daemon.
Launching a VxWorks Simulator Instance from the Command Line on page 29 Start a VxSim instance and an assign IP address to the target.
Viewing the Network and Boot Configurations on page 31 View the network configurations and boot parameters for a target.
Testing a Simulated Network on page 31 To test the simulated network, use the ping command to send test packets to the simulator instance from your host system.

1.1. Installing a WRTAP Driver

To simulate network IP connectivity to your host operating system, install a WRTAP driver.

About This Task

Once installed, the WRTAP driver is automatically initialized upon launching vxsim.

WNDRVR

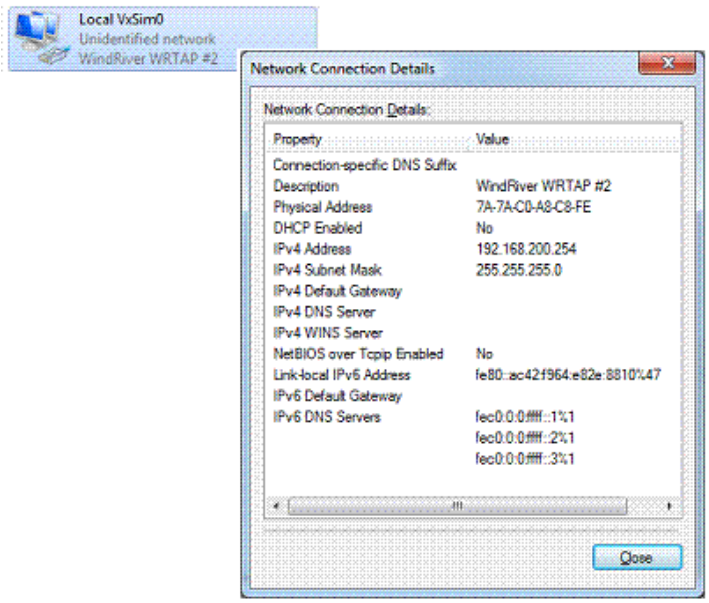
Procedure

1. Open a Windows command shell as an administrator.
2. In the command shell, use the following command syntax:

```
installDir\vxworks-7\host\x86-win32\bin\wrtapInstaller.exe install wrtap
```

where *installDir* is the name of your VxWorks installation directory.

You can view the new network connection present on your host from the network connections in the Windows Control Panel. The device type is Wind River WRTAP #*n*, where *n* can be 1, 2, 3, and so on. For example:



Parent topic: Full Network Simulation Using simnet on page 27

Parent topic: Networking Across Hosts with wrnetdlink Tutorial on page 60

1.2. Configuring and Starting the VxWorks Simulator Network Daemon

The VxSim provides a full network simulation environment through the VxSim network daemon.

About This Task

The daemon allows various protocol and subnet configurations.

For the purposes of this example, the default configuration file is used to configure a basic class C network subnet at 192.168.200.0.

Getting Help

To get help for vxsimnetd, type vxsimnetd -help at the host command shell prompt.

To get help for the vxsimnetd shell, type help at the vxsimnetd shell prompt.

For information on configuration files, see Creating a Network Daemon Configuration File on page 50.

For more information on vxsimnetd, see VxWorks Simulator Network Daemon Configuration and Use on page 40.

Procedure

1. Launch the VxSim using the default configuration file and access the vxsimnetd shell (-s option) with the following command:

```
> vxsimnetd -s
        vxsimnetd Debug Shell
vxsimnetd>
```

2. Display the current network configuration and status with the following commands:

```
vxsimnetd> subnet default
vxsimnetd> node default
vxsimnetd> packet default
```

   The *Device Name* in the output of the subnet command is the name of the network connection as it appears on your host (Local VxSim0 in this case).

   The default subnet has the following characteristics:

   - Subnet name: *default*
   - Address range: 192.168.200.0 to 192.168.200.255
     The subnet has 256 addresses.
   - Network access: External access.
     The subnet can communicate with the host.
   - Subnet mode: External Promiscuous
     The host can view every packet sent on the subnet. It allows you to attach a packet sniffer on the host interface to monitor traffic.

3. To quit the LAN simulation, use the quit command.

   For example:

```
vxsimnetd>quit
Stopping the Lan Simulation and then will do the 'quit'.
Please wait until <quit> operation is complete.....
>
```

Parent topic: Full Network Simulation Using simnet on page 27

1.3. Launching a VxWorks Simulator Instance from the Command Line

Start a VxSim instance and an assign IP address to the target.

Getting Help

To get help for the wrenv command, type wrenv -help at the development shell prompt.

To get help for the vxsim command, type vxsim -help at the development shell prompt.

WNDRVR

To get help for the kernel shell (->), type help at the kernel shell prompt.

For more information on vxsim, see VxWorks Simulator Options Reference on page 14.

For information on memory configuration, see Memory Configuration on page 76.

Procedure

1. Open a development shell either from Workbench 4 or from the command line.

   Use the wrenv command to set environment variables. For example:

```
installDir>wrenv -p vxworks-7
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.
installDir>
```

2. Access the default directory of the VIP. For example:

```
> installDir/workspace/my_vip/default
```

3. Enter the following command to launch the simulator instance with a simnet boot device (-d) with IP address 192.168.200.1 (-e):

```
>vxsim -d simnet -e 192.168.200.1
```

> **Note:** If you are issuing these command options from the command line on a Linux host, use double quotation marks around the option values. For example:
>
> ```
> vxsim -d "simnet" -e "192.168.200.1"
> ```

Results

The simulator memory configuration and the kernel shell prompt (->) appear in the terminal window. For example:

```
Checking License...OK
Virtual Base Address: 0x10000000
Virtual Top Address: 0x48000000 Virtual Size: 0x38000000 (896Mb)
Physical Base Address: 0x10000000
Physical Top Address: 0x18000000 Physical Size: 0x08000000 (128Mb)
Loading...
0x3740f0 + 0x3d9d0
...
...
Loading symbol table from host:
C:/vx7git/vxworks/workbench-4/workspace/vip_vsb_vx7_vxsim/default/vxWorks.sy
m ...done
->
```

Parent topic: Full Network Simulation Using simnet on page 27

WNDRVR

1.4. Viewing the Network and Boot Configurations

View the network configurations and boot parameters for a target.

Getting Help

To get help for the ifconfig command, enter ifconfig "" at the kernel shell prompt.

To get help for the VxWorks command shell (#), enter help at the command shell prompt.

Procedure

1. To view the network configuration for the target, use the ifconfig command from the kernel shell. For example:

```
-> ifconfig
ifconfig
...
simnet0Link type:Ethernet HWaddr 7a:7a:c0:a8:c8:01
inet 192.168.200.1 mask 255.255.255.0 broadcast 192.168.200.255
inet6 unicast fe80::787a:c0ff:fea8:c801%simnet0 prefixlen 64 automatic
UP RUNNING SIMPLEX BROADCAST MULTICAST
MTU:1500 metric:1 VR:0 ifindex:2
RX packets:56 mcast:0 errors:0 dropped:0
TX packets:71 mcast:8 errors:0
collisions:0 unsupported proto:0
RX bytes:2640 TX bytes:11k
value = 0 = 0x0
->
```

2. Use the show bootline command from the command shell to view the current boot parameters.

> **Note:** You can modify the boot parameters using the set bootline or bootChange commands.

Results

For information on boot parameters and how to configure boot parameters, see Boot Parameters on page 80.

Parent topic: Full Network Simulation Using simnet on page 27

1.5. Testing a Simulated Network

To test the simulated network, use the ping command to send test packets to the simulator instance from your host system.

Getting Help

To get help for the ping command, type ping at the host shell prompt.

Procedure

From a host command window or host shell, enter:

WNDRVR

```
> ping 192.168.200.1
```

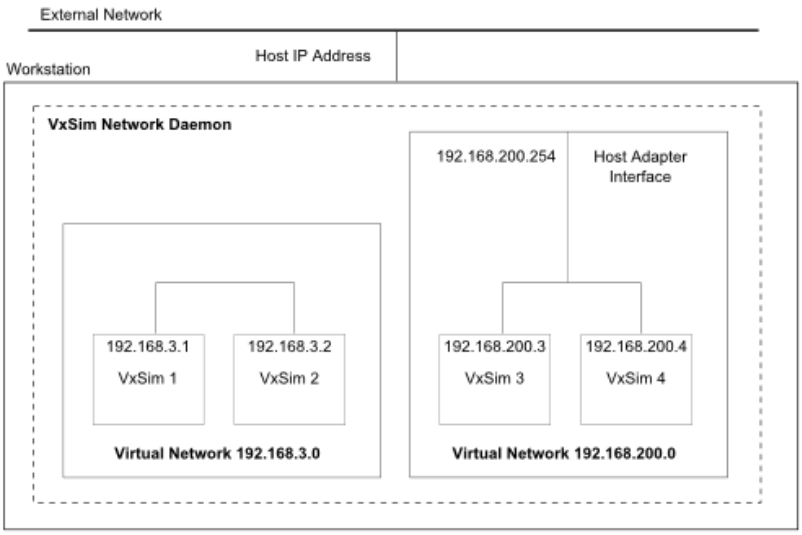
Parent topic: Full Network Simulation Using simnet on page 27

2. Building Network Simulations

Using the VxSim network daemon (vxsimnetd), you can link same-host simulator instances into simulated subnets.

By default, these internal subnets do not communicate with the host. However, included with the simulator is a simulated network drive—a host adapter interface—that you can use to give the host system an address on the simulated subnet.

The adapter on the simulated subnet lets you use the host to route packets for the subnet and thus link it with an external network. The following figure shows two subnets, 192.168.3.0 and 192.168.200.0, simulated on a single host.

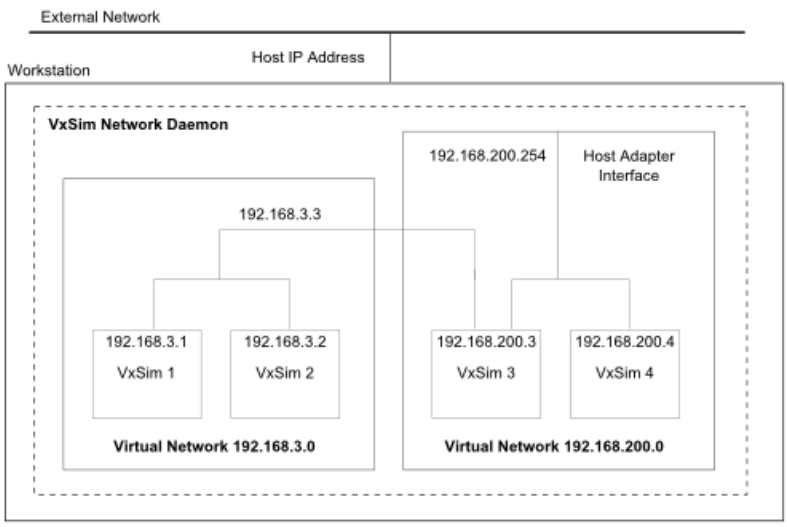


In addition, through the host adapter, packet sniffers such as tcpdump, snoop, or Wireshark can monitor traffic on an internally simulated subnet. Example usage of a packet sniffer in a simulated environment is provided in the *L2TP Programmer's Guide*.

Virtual subnet 192.168.3.0 is an entirely internal subnet. It is isolated from the host, although you can use a kernel shell to access a target. After you have access to one target on the subnet, you can use the subnet to communicate with other targets on the subnet. Virtual subnet 192.168.200.0 is an external subnet. It is networked to the host workstation through the host adapter interface at 192.168.200.254. If you set up the host system route table correctly, the host system can route packets for the 192.168.200.0 subnet. For more information, see Using Routes to Forward Packets Between Subnets on page 33.

As shown in the following figure, it is possible to create a multiple interface simulator instance. Using such a simulator instance, it is possible to route between simulated subnets.

WNDRVR



In this scenario, the multi-interface VxSim3 can route packets from the 192.168.3.0 subnet to the greater external network through the host adapter to the host, which can then route packets to the external network through its network interface. Example usage of multiple network instances is provided in Using Multiple Network Interfaces on page 33.

3. Using Multiple Network Interfaces

You can use multiple network interfaces to create a multi-subnet environment.

To configure a simulator instance with multiple network interfaces, vxsim includes the -ni option. The syntax for this option is as follows:

> **vxsim -ni** *interfaceNameInterfaceNumberNum* **[=***ipAddr***] : [***ipMask***]**

For example:

```
>vxsim -ni simnet0=192.168.200.1
```

The default network mask is 255.255.255.0.

To create multiple network interfaces with a single command, you can chain the descriptions using a semi-colon (;) as a delimiter. For example:

```
>vxsim -ni simnet0=192.168.200.1;simnet1=192.168.3.1;simnet2=192.168.4.1
```

This command starts a simulator instance configured with three simulated network interfaces.

For example usage of the -ni command in a simulated multi-subnet environment that includes a router, see the basic networking example in the *VxWorks 7 Network Stack Programmer's Guide*.

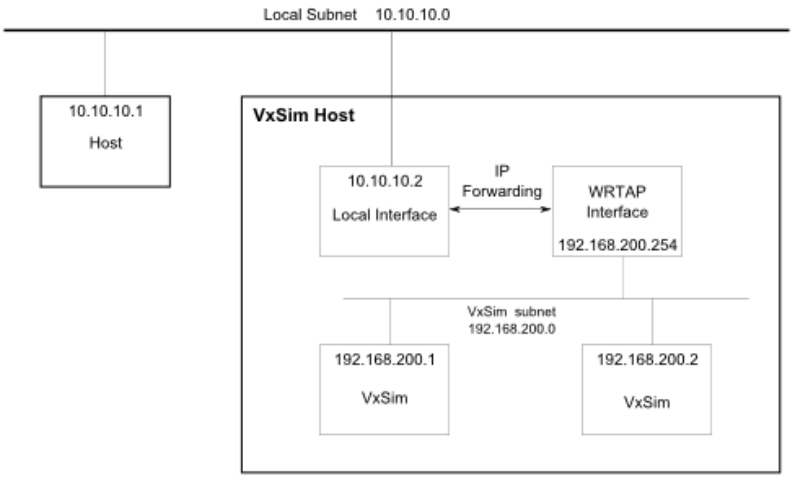
4. Using Routes to Forward Packets Between Subnets

Integrate the VxSim directly into your local network with IP forwarding.

About This Task

By default, the VxSim network daemon (vxsimnetd) configures the simulated host network interface with IP address 192.168.200.254.

In the diagram below, the host appears as a gateway between two subnets: the local subnet at 10.10.10.0 and the simulator subnet at 192.168.200.0.



To enable the configuration of routing tables for IP forwarding, you must add the **INCLUDE_ROUTECMD** component to your VIP. See the discussion of *Additional VIP Components* in Kernel Configuration Components (VIP) on page 5.

In this environment, if you want to access host 10.10.10.1 from the simulator, you must set up IP forwarding on your host and enable the following routes with the route command:

Getting Help

To get help for the vxsim command, type vxsim -help at the development shell prompt. For more information on vxsim, see VxWorks Simulator Options Reference on page 14.

To get help for the route command, type route at the host shell or at the target command shell prompt, or type routec at the kernel shell prompt.

Procedure

1. Enable IP forwarding on the host that will be running the simulator instance.
   - On Windows hosts, start the **Routing and Remote Access** service.
   - On Linux hosts, run the following command with root permissions:

     ```
     $ # echo "1" > /proc/sys/net/ipv4/ip_forward
     ```

     or, add the following line to **/etc/sysconfig/network**

     ```
     FORWARD_IPV4=yes
     ```

     and restart the host.
2. To configure a route from the 10.10.10.0 subnet through the host interface at 192.168.200.254, do one of the following:
   - Specify 192.168.200.254 as the gateway in the vxsim boot parameters (vxsim -g 192.168.200.254)

- Use the following command from the target command shell:

```
-> cmd
[vxWorks *]#route add -net 10.10.10.0/24 192.168.200.254
```

3. Verify the route configuration with the following command:

```
[vxWorks *]#route show
```

4. On each remote Windows host, indicate that host 10.10.10.2 is the gateway to reach the 192.168.200.0 network. For example, execute the following from your host command shell:

```
C:\> route add 192.168.200.0 mask 255.255.255.0 10.10.10.2
```

Note: If you are performing this procedure on a Linux host, execute the following from your host command shell:

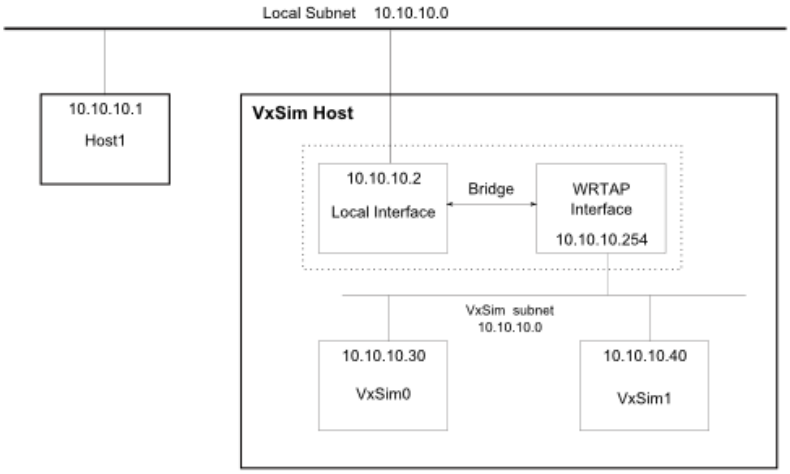
```
% route add -net 192.168.200.0 gw 10.10.10.2
```

You can now reach the simulator instance from the remote host.

5. Using a Bridge to Connect Subnets

To simplify route settings, you can connect the VxSim directly to your local network through a bridge—configured on your host machine—between the local network interface and the simulated host network interface

The goal of setting up such a bridge is to allow a simulator instance started on host 10.10.10.2 to ping any host on the local subnet without having to set up additional routes.



Getting Help

To get help for the vxsim command, type vxsim -help at the development shell prompt.

To get help for vxsimnetd, type vxsimnetd -help at the command shell prompt.

To get help for netstat, type netstat -help at either the host or target command shell prompt.

For more information on vxsim, see VxWorks Simulator Options Reference on page 14.

For more information on vxsimnetd, see VxWorks Simulator Network Daemon Configuration and Use on page 40.

To start vxsimnetd as a service, see Running the Network Daemon as a Service—Windows on page 40, Running the Network Daemon as a Service on Linux on page 43, or Running the Network Daemon as a Service on Ubuntu on page 45.

Configuring a Bridge on a Windows Host on page 36 On a Windows host, connect a simulator directly to your local network through a bridge.
Configuring a Bridge on a Linux Host on page 37 On a Linux host, connect a simulator directly to your local network through a bridge.

5.1. Configuring a Bridge on a Windows Host

On a Windows host, connect a simulator directly to your local network through a bridge.

Procedure

1. Create a vxsimnetd configuration file for your local subnet. For example, if your local subnet is 10.10.10.0:

```
SUBNET_START local_subnet {
    SUBNET_ADDRESS = "10.10.10.0";
    SUBNET_MASK = "255.255.255.0";
    SUBNET_EXTERNAL = yes;
    SUBNET_EXTPROMISC = no;
};
```

2. Start vxsimnetd using the configuration file that you created above.
3. Confirm the network interfaces in Network Connections.

   The WRTAP interface is configured on the same subnetwork as your local network interface. For example, you should see a local interface of 10.10.10.2 and a WRTAP interface of 10.10.10.254.

4. To set up the bridge, select both interfaces and then right-click and select **Add to Bridge** from the pop-up menu.
5. Configure the bridge in the same way the local interface is configured (either DHCP or fixed address and gateway).
6. Start a simulator instance from a development shell. For example:

```
> vxsim p 0 -d simnet -e 10.10.10.30
```

   You should now be able to access any other host without modifying routing tables.

7. Check your routes using the netstat command from your host and the kernel shell:

```
C:\> netstat -r
-> netstat -r
```

8. Ping Host1, as shown in the diagram in Using a Bridge to Connect Subnets on page 35, from VxSim0 as follows:

```
-> cmd
[vxWorks *]# ping 10.10.10.1
```

   If you started vxsimnetd as a service with your local subnet configuration, the bridge should continue to be available after a reboot of your host.

WNDRVR

Note: When vxsimnetd is stopped, it removes the TAP interface from the bridge. When vxsimnetd is restarted, you must run the following commands to install the TAP interface into the bridge:

```
$ brctl addif bridge0 tap1
$ ifconfig tap1 0.0.0.0
```

Parent topic: Using a Bridge to Connect Subnets on page 35

5.2. Configuring a Bridge on a Linux Host

On a Linux host, connect a simulator directly to your local network through a bridge.

About This Task

These steps must be performed locally on the host machine and not through a remote connection. Any remote connection will be temporarily disrupted during bridge set up.

Note: To use the simulator on a local bridge in Linux, you must ensure that the bridge module is installed on your Linux host. You can verify this using the modinfo bridge command. If necessary, install the package that contains this module. In addition, you will need the brctl command that is available in the bridge-utils package.

Procedure

1. To make sure the bridge module is available and loaded on your Linux host, issue the following commands:

```
# modinfo bridge
filename: /lib/modules/2.6.9-67.0.20.ELhugemem/kernel/net/bridge/bridge.ko
license: GPL
vermagic: 2.6.9-67.0.20.ELhugemem SMP 686 REGPARM 4KSTACKS gcc-3.4
depends: atm
# modprobe bridge
```

2. Once you have verified that the required modules are available, create a vxsimnetd configuration file for your local subnet (10.10.10.0 in this example):

```
SUBNET_START local_subnet {
    SUBNET_ADDRESS = "10.10.10.0";
    SUBNET_MASK = "255.255.255.0";
    SUBNET_EXTERNAL = yes;
    SUBNET_EXTPROMISC = no;
    SUBNET_EXTDEVNUM = 1;
};
```

3. Start vxsimnetd with this configuration file or source it through an existing vxsimnetd shell.

   From a command shell with root privileges, issue the following command:

```
$ vxsimnetd -f local_subnet.conf -sv
```

   If vxsimnetd was previously started as root with a shell server, issue the following commands from a command shell:

WNDRVR

```
$ cp local_subnet.conf /tmp/local_subnet.conf
$ telnet localhost 7777
vxsimnet> source /tmp/local_subnet.conf
subnet(s) <local_subnet> added.
```

4. With root privileges, set up the bridge as follows.

```
$ brctl addbr bridge0
$ brctl addif bridge0 eth0
$ brctl addif bridge0 tap1
$ ifconfig tap1 0.0.0.0
$ ifconfig eth0 0.0.0.0
$ ifconfig bridge0 10.10.10.2 netmask 255.255.255.0 up
```

5. From the VxWorks development shell, start the simulator instance using the following command:

```
-> vxsim p 0 -d simnet -e 10.10.10.30
```

6. Check your routes using the netstat command from your host and the target command shell:

```
$ netstat -r
# netstat -r
```

7. Ping Host1, as shown in the diagram in Using a Bridge to Connect Subnets on page 35, from VxSim0 as follows:

Parent topic: Using a Bridge to Connect Subnets on page 35

6. Starting a Simulator Instance without an IPv4 Address

You can start a simulator instance and attach to a subnet through its name.

About This Task

Procedure

Choose one of the following strategies for starting a simulator instance.

Option	Description

Start a simulator that attaches to the first configured subnet	`>vxsim -d simnet`
	`>vxsim -ni simnet`
	
	These commands start a simulator that attaches to the first configured subnet (neither IPv4 address is specified). In this example, a MAC address can no longer be deduced from the IP address so a node number is used instead. The first attaching simulator instance gets 7a:7a:0:0:0:1, and the second instance gets 7a:7a:0:0:0:2, where 7a:7a is the subnet MAC prefix of the first configured subnet.

WNDRVR

Option	Description
	Note: In this example, the MAC address is no longer fixed and can change if the simulator instance is rebooted. This may cause a problem with ARP tables.
Start a simulator instance that attaches to a named subnet	`>vxsim -ni simnet0:default` Use this command to start a simulator instance that attaches to a subnet named **default**. The MAC address is determined as described in the earlier example.

Start a simulator instance with a simulated subnet interface.	You can get a fixed MAC address by specifying an IPv4 address that is not used to configure the simnet interface if the **INCLUDE_NET_BOOT_CONFIG** component is not defined.
	`>vxsim -d simnet -e 192.168.3.1`
	`>vxsim -ni simnet1=192.168.3.1`
	This command sequence starts a simulator instance with a simulated subnet interface with a MAC address of 7a:7a:c0:a8:03:01 and an IP address (or addresses) that can be configured later using the ifconfig command.

A simulator instance can also be configured with multiple network interfaces using the -ni option. For more information on this configuration, see Using Multiple Network Interfaces on page 33.

7. Using IPv6 in a Simulated Network Environment

For an example usage of IPv6 in a simulated network environment, see *VxWorks 7 Network Stack Programmer's Guide*.

WNDRVR

6. VXWORKS SIMULATOR NETWORK DAEMON

1. VxWorks Simulator Network Daemon Configuration and Use

The VxSim includes a network daemon that you can use to link multiple simulator instances into one or more subnets. You can also link these subnets (or even individual simulator instances) to the larger Internet.

VxWorks Simulator Network Daemon (vxsimnetd)

The VxWorks simulator provides a full network simulation environment through the VxWorks simulator network daemon (vxsimnetd). The VxWorks simulator network daemon allows you to select simnet as a boot device for the simulator instance, define a simulated network based on a configuration file, and view and modify the network configuration from the command line. The vxsimnetd utility can be run as a Windows service or a root service on Linux, or from the command line to provide full network support for the VxWorks simulator without the need for administrator or root privileges on the host system.

The network daemon can support any protocol over the Ethernet layer (for example, TCP/IP). Thus, you can use the simulator instances to test any broadcasting or multicasting features you may have built into an application.

Note:

If you want to use a simulator instance(s) on a simulated network, you must start the network daemon before you start the simulator instance. Keep in mind that even a connection between the host system and a single instance requires the network daemon.

This section discusses how to configure your system and the required simulator instances for use in a simulated subnet or subnets. It includes general network simulation information, instructions for setting up the VxSim network daemon (vxsimnetd), installing host connection drivers, and information on configuring your simulated network.

2. Running the Network Daemon as a Service—Windows

Wind River recommends starting vxsimnetd as a Windows service because this method provides full network support for the VxSim even if you are not logged in with administrator or root privileges on the host system.

Note:

You must remove any previously installed network daemon services before attempting to install a new service. This may be the case even if you removed your previous installation using a Wind River uninstallation utility. For information on removing the network daemon service, see Removing the Network Daemon Service on page 43.

On Windows hosts, you can install and configure the network daemon service from the VxWorks development shell (recommended) or from the Windows command shell.

Note:

The **vxsimnetds_inst.exe** program is a command-line installer for the vxsimnetd service. Run it just once, to install the service, or again to modify the parameters of the service.

To run vxsimnetd directly from the command line (that is, not as a service), use **vxsimnetd.exe**.

Installing the Service from the VxWorks Development Shell on page 41 Install the network daemon from the VxWorks development shell.
Installing the Service from the Windows Command Shell on page 41 Install the network daemon from the Windows command shell.
Configuring the Network Daemon Service on page 42 Configure the network daemon service from the command line.

Removing the Network Daemon Service on page 43 Remove the network daemon service (vxsimnetds) from the command line.

2.1. Installing the Service from the VxWorks Development Shell

Install the network daemon from the VxWorks development shell.

Procedure

1. Open the VxWorks development shell.
2. If you have an existing vxsimnetd service from a previous installation, you must remove the service. To remove it, execute the following command from the VxWorks development shell:

```
> vxsimnetds_inst.exe /u
```

3. To install the new network daemon, execute the following command from the VxWorks development shell:

```
> vxsimnetds_inst.exe
```

Parent topic: Running the Network Daemon as a Service—Windows on page 40

2.2. Installing the Service from the Windows Command Shell

Install the network daemon from the Windows command shell.

Procedure

1.

> Note: You must run **vxsimnetds_inst.exe** with administrator privileges.

Log in to the Windows host with administrator privileges.

2. If you have an existing vxsimnetd service from a previous installation, you must remove the service. To remove it, use the following command syntax:

```
C:\> installDir\vxworks-7\host\x86-win32\bin\vxsimnetds_inst.exe /u
```

where *installDir* is the name of your VxWorks installation directory.

3. Install the new network daemon using the following command syntax:

```
C:\> installDir\vxworks-7\host\x86-win32\bin\vxsimnetds_inst.exe
```

4. Start the service:
   1. From Administrative Tools, access **Services**.
   2. Select **Wind River Network Daemon for VxWorks Simulator** and set the startup type to **Automatic**.

   You can then start the service.

   By default, the network daemon starts with the default 192.168.200.0 external subnet configuration and with a shell server (-sv option). To change these options, right-click **Wind River Network Daemon for VxWorks Simulator**, select **Properties**, and specify the desired start parameters before starting the service.

WNDRVR

Once the network daemon service is started, non-administrator users can start simulator instances and attach them to any configured subnet.

Parent topic: Running the Network Daemon as a Service—Windows on page 40

2.3. Configuring the Network Daemon Service

Configure the network daemon service from the command line.

Procedure

The basic command-line execution required to install the network daemon service is as follows:

```
C:\> installDir\vxworks-7\host\x86-win32\bin\vxsimnetds_inst.exe
Install Wind River Network Daemon for VxWorks Simulator from
C:\vx7git\vxworks\vxworks-7\host\x86-win32\bin\vxsimnetds.exe
Wind River Network Daemon for VxWorks Simulator installed.
C:\>
```

To get help, use the following command:

```
> vxsimnetds_inst.exe -help
```

The following configuration options are available:

/p

The preserve option (/p) allows you to keep a previous network daemon service (vxsimnetds) configuration. When used, this option loads the saved configuration parameters from the previously installed network daemon service (parameters are saved when a service is removed). If the /p option is omitted, default parameters are used.

/d

The /d option specifies the directory to which the vxsimnetds files are copied. This option is mandatory if you are installing on a mounted drive. Note that this option requires that you specify an existing directory; the directory cannot be created at runtime.

/u

The /u option stops and uninstalls the current vxsimnetds service. When the service is removed, the configuration parameters are saved and can be reused for the next installation by specifying the /p option. (For more information, see Removing the Network Daemon Service on page 43.)

/get *parameter*

This option allows you to display the following network daemon service parameters: CONFIG_FILE, SHELL_SERVER, SHELL_SERVER_PORT, LOGGING_LEVEL, and LOG_FILE.

Specifying this option without any parameters displays values for all parameters in the following form:

```
Subnet Configuration File: file
Shell Server: Enabled
Shell Server Port Number: 7777
Logging Level: 1
Log File: file
```


WNDRVR

/set *parameter*

This option allows you to set the following network daemon service parameters: ConfigFile, ShellServer, ShellPort, LogLevel, and LogFile.

Parameters can be set as follows:

```
C:\>vxsimnetds_inst /set ConfigFile=file
/set ShellServer=[yes|no]
/set ShellPort=value
/set LogLevel=value
/set LogFile=logFile
```

Parent topic: Running the Network Daemon as a Service—Windows on page 40

2.4. Removing the Network Daemon Service

Remove the network daemon service (vxsimnetds) from the command line.

Procedure

Open a VxWorks development shell and enter the following command:

```
-> vxsimnetds_inst.exe /u
Stopping Wind River Network Daemon for VxWorks Simulator.....
Wind River Network Daemon for VxWorks Simulator stopped.
Wind River Network Daemon for VxWorks Simulator removed.
```

Note: The product uninstaller does not uninstall vxsimnetd. If you plan to uninstall the product, be sure to uninstall vxsimnetd (as described above) before running the uninstaller program.

Results

The command stops the network daemon and removes the vxsimnetd service.

Parent topic: Running the Network Daemon as a Service—Windows on page 40

3. Running the Network Daemon as a Service on Linux

Run vxsimnetd as a root service on Linux hosts other than Ubuntu.

About This Task

To do this, create scripts for the systems that start vxsimnetd automatically on reboot (use the -sv option if you want the ability to modify network configuration).

Note: The following steps must be executed with root privileges.

Install vxsimnetd as a root service and start the service using the following steps:

Procedure

1. Copy **vxsimnetd** to your **/usr/sbin** directory. For Linux hosts, **vxsimnetd** is located in *installDir* **/vxworks-7/host/x86-linux2/bin/**.
2. Create a vxsimnetd script as follows in **/etc/init.d**:

```
#!/bin/sh
#
# chkconfig: 3 91 02
# description: Starts and stops the vxsimnetd daemon \
#       used to provide VxWorks Simulator network services.
#
# pidfile: /var/run/vxsimnetd.pid


# Source function library.
if [ -f /etc/init.d/functions ] ; then
        . /etc/init.d/functions
elif [ -f /etc/rc.d/init.d/functions ] ; then
        . /etc/rc.d/init.d/functions
else
exit 0
fi

# Check that vxsimnetd exists.
[ -x /usr/sbin/vxsimnetd ] || exit 0

RETVAL=0

start() {
    echo -n $"Starting vxsimnetd service: "
        daemon vxsimnetd -sv
    RETVAL=$?
    echo
    [ $RETVAL -eq 0 ] && touch /var/lock/subsys/vxsimnetd || \
                RETVAL=1
    return $RETVAL
}

stop() {
    echo -n $"Shutting down vxsimnetd services: "
    killproc vxsimnetd
    RETVAL=$?
    echo
    [ $RETVAL -eq 0 ] && rm -f /var/lock/vxsimnetd
    echo ""
    return $RETVAL
}

restart() {
    stop
    start
}

rhstatus() {
    status vxsimnetd
}
```

```
case "$1" in
    start)
        start
        ;;
    stop)
        stop
        ;;
    restart)
        restart
        ;;
    status)
        rhstatus
        ;;
    condrestart)
        [ -f /var/lock/subsys/vxsimnetd ] && restart || :
        ;;
    *)
        echo $"Usage: $0 {start|stop|restart|status|condrestart}"
        exit 1
esac

exit $?
```

3. Create a link.

   Run the following command in **/etc/init.d/**.

   ```
   $ /sbin/chkconfig --add vxsimnetd
   ```

   This creates two links on **/etc/init.d/vxsimnetd**, **/etc/rc3.d/S91vxsimnetd** and **/etc/r6.d/K02vxsimnetd**.

4. Start vxsimnetd as follows:

   ```
   # /etc/init.d/vxsimnetd start
   ```

   or reboot your host.

4. Running the Network Daemon as a Service on Ubuntu

Run vxsimnetd as a root service on Ubuntu hosts.

About This Task

To do this, create scripts for the systems that start vxsimnetd automatically on reboot (use the -sv option if you want the ability to modify network configuration).

Note: The following steps must be executed with root privileges.

Install vxsimnetd as a root service and start the service using the following steps:

Procedure

1. Copy **vxsimnetd** to your **/usr/sbin** directory. For Linux hosts, **vxsimnetd** is located in *installDir* **/vxworks-7/host/x86-linux2/bin/**.
2. Create a vxsimnetd script as follows in **/etc/init.d**:

```
#!/bin/sh
#
# chkconfig: 3 91 02
# description: Starts and stops the vxsimnetd daemon \
#       used to provide VxWorks Simulator network services.
#
# pidfile: /var/run/vxsimnetd.pid


# Source function library.
. /lib/lsb/init-functions

# Check that vxsimnetd exists.
[ -x /usr/sbin/vxsimnetd ] || exit 0
RETVAL=0
start() {
    echo -n $"Starting vxsimnetd service: "
    start_daemon /usr/sbin/vxsimnetd -sv -force
    RETVAL=$?
    echo
    [ $RETVAL -eq 0 ] && touch /var/lock/vxsimnetd || RETVAL=1
    return $RETVAL
}

stop() {
    echo -n $"Shutting down vxsimnetd services: "
    killproc vxsimnetd
    RETVAL=$?
    echo
    [ $RETVAL -eq 0 ] && rm -f /var/lock/vxsimnetd
    echo ""
    return $RETVAL
}

restart() {
    stop
    start
}

rhstatus() {
    status_of_proc vxsimnetd
}
case "$1" in
    start)
        start
        ;;
    stop)
        stop
        ;;
    restart)
        restart
```

```
        ;;
    status)
        rhstatus
        ;;
    condrestart)
        [ -f /var/lock/subsys/vxsimnetd ] && restart || :
        ;;
    *)
        echo $"Usage: $0 {start|stop|restart|status|condrestart}"
        exit 1
esac

exit $?
```

3. Create a link.

   Run the following command in **/etc/init.d/**.

   ```
   $ update-rc.d vxsimnetd defaults
   ```

   This creates two links on **/etc/init.d/vxsimnetd**, **/etc/rc3.d/S91vxsimnetd** and **/etc/r6.d/K02vxsimnetd**.

4. Start vxsimnetd as follows:

   ```
   # /etc/init.d/vxsimnetd start
   ```

   or reboot your host.

5. Starting the Network Daemon from the Command Line

If you do not install and configure the VxWorks simulator network daemon as a service, you can use the vxsimnetd command to manually start the network daemon on your host system.

About This Task

You can configure the daemon using a configuration file statically at start-up time or you can configure it interactively using the network daemon shell. You can also combine these configuration methods, which allows you to use a configuration file to supply some defaults and read in additional configuration files as needed.

> Note: If the daemon must support an externally visible subnet, you must launch the daemon from a task with the appropriate privileges. On Linux hosts, this means starting the daemon with supervisor or root privileges. On a Windows host, this means starting the daemon with administrator privileges.

> Note: vxsimnetd can also be started without any configuration options. In this case, the network daemon is started with a default external subnet of 192.168.200.0 and with the host node set in promiscuous mode. The default configuration is defined in *installDir* **/vxworks-7/host/resource/vxsim/vxsimnetd.conf**.

The vxsimnetd command supports the following options:

WNDRVR

-f or -file

This option specifies the configuration file that is parsed when the network daemon starts. For more information on the format of this file, see Creating a Network Daemon Configuration File on page 50.

-s or -shell

This option starts a shell that you can use to control network daemon configuration interactively. For more information on the shell options, see Network Daemon Shell Reference on page 48.

-sv or -shellserver

This option starts the network daemon in server/background mode. When in background mode, you can telnet to a debug port to access the shell. The -sv and -s options are mutually exclusive.

-sp or -shellport

This option specifies the debug port used to start a shell on a network daemon in background mode. If not specified, the default port is 7777.

-t or -test

Tests the specified network configuration file (***.conf**).

-v or -version

Displays the version of the network daemon.

-force

Forces the deletion of IPC objects left after vxsimnetd dies (UNIX only).

Procedure

Choose one of the following options to configure vxsimnetd statically or dynamically.

- To configure the daemon statically, use a command such as the following, where **vxsimnetd.conf** is a file supplying configuration parameters.

  ```
  % vxsimnetd -f vxsimnetd.conf
  ```

- To start the VxWorks simulator network daemon interactively, use a command such as the following, where **vxsimnetd.conf** is a file supplying configuration parameters:

  ```
  % vxsimnetd -f vxsimnetd.conf -s
  ```

  If you use the -sv option instead of the -s option, the shell runs in the background and is accessible using telnet. For example:

  ```
  % telnet hostname portNumber
  ```

  portNumber defaults to 7777, but vxsimnetd supports the -sp parameter, which you can use to specify a different port number.

6. Network Daemon Shell Reference

You can access the network daemon shell by starting vxsimnetd with the -s (or -shell) or -sv (or -shellserver) option.

The shell supports command-line completion as well as history with two editing modes, emacs (default) or vi.

The available shell commands are:

subnet *subnetName]*

This command displays subnet information. When no subnet is specified, the command lists a summary for all configured subnets. A detailed summary is provided when a subnet name is specified.

node *subnetName* [*nodeIp* | *nodeNb*]

This command displays information about how many nodes are configured and used. For example:

```
vxsimnetd> node default
NODE INFORMATION:
        CONFIGURED IN-USE     MAX        TOTAL      FAIL
        33         1          1          1          0

Current Nodes of the subnet (default):
#   COMM       STATUS IP              PROMISC RCVQ PID
0   special(*) UP     192.168.200.254 Yes     0    24076
```

If you specify a node number (*nodeNb*) or node IP address (*nodeIp*), the node command displays specifics about the particular node, such as the number of packets sent and received. For example:

```
vxsimnetd> node default 0
#   COMM       STATUS IP              PROMISC RCVQ PID
0   special(*) UP     192.168.200.254 Yes     0    24076

MAC ADDRESS:
        Mac Address               7a:7a:c0:a8:c8:fe

SEND/RECEIVE STATISTICS:
        # of receives             0
        # of sends                6
        # of send failures        6

RECEIVE QUEUE INFORMATION:
        CONFIGURED CURRENT    MAX
        64         0          0
```

packet *subnetName*

This command displays packet information for a subnet. For example:

```
vxsimnetd> packet default
PACKET INFORMATION:
        CONFIGURED IN-USE     MAX        TOTAL      FAIL
        100        1          1          7          0

HANGING PACKETS:
PKT-# SEND-NODE          RECV-NODE        LEN        REFCNT    PKTPTR
0     192.168.200.254[E] N/A              1514       0         0xff0e2b14
```

In this example, the default subnet is configured with 100 packets, only one is currently in use, and seven packets were used in total. The hanging packets section displays packets that are allocated but not yet sent.

help *command*

This command specifies detailed help for a given command. If no command is specified, a summary of all available commands is provided.

?

Displays a one-line summary for all commands.

WNDRVR

quit

This command exits the shell. If you started vxsimnetd with the -s option, this command destroys all subnets and vxsimnetd exits. For more information on vxsimnetd options, see VxWorks Simulator Network Daemon Configuration and Use on page 40.

source *configFile*

This command reads subnet configuration information from a file and adds the corresponding subnets. If vxsimnetd cannot add all configured subnets, then this command adds no subnets at all.

delete *subnetName*

This command deletes a configured subnet. To delete all subnets, use delete all.

extpromisc *subnetName* [0 | 1]

This command sets the promiscuous mode for the host node of an external subnet. The 0 option sets promiscuous mode to off, 1 sets promiscuous mode to on. When the external node is in promiscuous mode (1), it receives every packet sent on the subnet. While this heavily impacts network performance, it allows you to analyze network traffic by connecting a packet sniffer on the external host node interface.

errorrate *subnetName*

This command sets the error rate for a given subnet. The error rate is the percentage of packets that will not be sent without giving error notification to the sender. Thus, if the error rate is set at five percent, five randomly chosen packets per 100 will be purposely lost. This feature is provided to simulate packet loss on an actual subnet.

timeout *subnetName*

This command sets the subnet timeout value. If a node does not read any packets for the length of time specified as the timeout, the packets are picked up by garbage collection.

mode [vi | emacs]

This command sets the shell editing mode to vi or emacs.

7. Creating a Network Daemon Configuration File

A network daemon configuration file specifies values for the vxsimnetd configuration parameters.

To specify a configuration file, use the -f option with the vxsimnetd command (the command used to start the network daemon). To use the default configuration file, omit the -f option. The default configuration is stored in *installDir* **/vxworks-7/host/resource/vxsim/vxsimnetd.conf**.

In your configuration file, to assign a value to a parameter, enter a semicolon-terminated (;) line with the following format, where *PARAMETER* is either a parameter name in capital letters or an alias:

```
PARAMETER = value ;
```

For parameters related to a subnet, group those parameters using the following syntax:

```
SUBNET_START subnetName {
    SUBNET_PARAM = value ;
};
```

For example, consider the following default configuration file:

WNDRVR

```
SUBNET_START default {
    SUBNET_EXTERNAL = yes;
    SUBNET_EXTPROMISC = yes;
    SUBNET_ADDRESS = "192.168.200.0";
};
```

This configures the VxWorks simulator network daemon to support a subnet with external access. The network address for the subnet is 192.168.200.0 and, because the network mask is not specified, the default mask applies. For 192.168.200.0, that would be the mask for a class C address, which is 0xffffff00. The address range is thus 192.168.200.0 to 192.168.200.255.

To add another subnet, you could add the following lines:

```
SUBNET_START user1 {
    SUBNET_UID = 323;
    SUBNET_GID = 100;
    SUBNET_ACCESSMODE = "0600";
    SUBNET_ADDRESS = "192.168.201.0";
};
```

Note:

While you can use a configuration file to overwrite the default values for these parameters, if you add the configuration file dynamically, through the vxsimnetd shell, the default values are not modified.

For example, if vxsimnetd is started with a configuration in which DEFAULT_MAC_PREFIX is set to **7a7b**, every subsequent dynamically added subnet will use this value. To add a subnet with a different value, you must add it through a configuration file that redefines the value of DEFAULT_MAC_PREFIX.

For more information about the parameters supported in a VxWorks simulator network daemon configuration file, see Network Daemon Configuration File Parameters Reference on page 51.

Network Daemon Configuration File Parameters Reference on page 51 The VxSim network daemon configuration file supports a list of parameters.

7.1. Network Daemon Configuration File Parameters Reference

The VxSim network daemon configuration file supports a list of parameters.

The parameters supported in a VxSim network daemon configuration file are described in the following tables.

Default Parameters

Parameter	Description
DEFAULT_GARBAGE	Alias: dgarbage Default value: 30 Specifies the number of seconds in the garbage collection interval. For each subnet, the garbage collection thread runs every DEFAULT_GARBAGE seconds. Garbage collection removes any dead nodes from the configured subnets.
DEFAULT_MACPREFIX	Alias: dmacprefix Default value: 7a:7a Specifies the first bytes of simulator Ethernet addresses.

Parameter	Description
DEFAULT_UID	Alias: duid Default value: user ID of the user that started the network daemon Defines the user ID (Linux only).
DEFAULT_GID	Alias: dgid Default value: group ID of the user that started the network daemon Defines the group ID (Linux only).
DEFAULT_ACCESSMODE	Alias: daccessmode Default value: "0666" Defines access mode (Linux only). You can use the three parameters (duid, dgid, and daccessmode) to restrict access to subnets to a given user or group of users when the network daemon is shared between users on the same host.
DEFAULT_EXTERNAL	Alias: dexternal Default value: no Defines the default subnet type.
DEFAULT_EXTPROMISC	Alias: dextpromisc Default value: yes Defines whether the external subnet host node is set in promiscuous mode.
DEFAULT_ERRORRATE	Alias: derrorrate Default value: 0 Defines the default subnet error rate.
DEFAULT_TIMEOUT	Alias: dtimeout Default value: -1 Defines how long packets queued to a simulator instance are left in the queue. The default is forever.
DEFAULT_FILTERED_ MCAST_IN	Specifies up to 5 multicast MAC addresses that vxsimnetd will not forward from external interface to underlying subnet. *DEFAULT_FILTERED_MCAST_IN =* 01:00:5e:28:c9:01,01:00:5e:28:c9:02;

Resource-Related Parameters

Parameter	Description
SUBNET_ADDRESS	Alias: address Default value: "0.0.0.0"

Parameter	Description
	Specifies the network address for this subnet.
SUBNET_MASK	Alias: mask Default value: Pre-CIDR mask associated with the address in SUBNET_ADDRESS. Specifies the subnet mask for this subnet.
SUBNET_EXTERNAL	Alias: external Default value: DEFAULT_EXTERNAL Specifies whether this subnet can communicate with the host on which it runs. This communication requires you to create a simulator target with a network interface on the host system's network, and to start the VxWorks simulator network daemon with administrator privileges.
SUBNET_EXTPROMISC	Alias: extpromisc Default value: DEFAULT_EXTPROMISC Specifies whether the host sees every packet sent on this subnet. It allows you to attach a sniffer on the host interface to monitor traffic. However, it has a dramatically negative impact on network performance.

Option Parameters

Parameter	Description
SUBNET_BROADCAST	Alias: broadcast Default value: yes Specifies whether to allow MAC broadcast packets.
SUBNET_ERRORRATE	Alias: errorrate Default value: DEFAULT_ERRORRATE Defines the subnet error rate (the percentage of packet loss on this subnet)
SUBNET_TIMEOUT	Alias: timeout Default value: DEFAULT_TIMEOUT Defines how long packets that are queued are left in the queue before garbage collection removes them.
SUBNET_MTU	Alias: mtu Default value: 1500 Defines the MTU value that a simulator instance is configured to use when it attaches to a subnet.
SUBNET_EXTCONNNAME	Alias: extonnname Default value:

Parameter	Description
	Specifies the network interface name to use for this subnet as set in **Control Panel > Network Connections** (Windows only).
SUBNET_EXTDEVNUM	Alias: extdevnum Default value: 0 Specifies the host device number to use on Linux. This parameter is required when using more than one external subnet.

Parent topic: Creating a Network Daemon Configuration File on page 50

8. Multiple External Subnet Caveats

Caveats must be observed when configuring the VxWorks simulator network daemon with multiple external subnets.

On Windows hosts

You must install a WRTAP host connection driver for each external subnet. You may also want to specify a name for the device driver used by a given subnet through the SUBNET_EXTCONNNAME configuration parameter.

On Linux hosts

You must specify the device number of all but the first external subnet using the SUBNET_EXTDEVNUM parameter.

7. HOST CONNECTION DRIVERS

1. Host Connection Drivers—TAP/WRTAP

The host connection driver allows you to set up a subnet that can communicate with your host.

To simulate the network IP connectivity of a VxWorks target on Windows, the VxWorks simulator includes a host connection driver (WRTAP driver) that allows VxWorks to communicate with your host and other IP-based devices by exchanging IP packets with the host operating system. Once installed, WRTAP drivers appear as network connections on your host and are automatically initialized upon launching vxsim.

On Linux, the TAP driver provides the simulated network connectivity of a VxWorks target.

You only need to install this driver if you want to set up an externally visible subnet that can communicate with the host of simulator instances.

Once installed, you can use vxsimnetd to configure its IP address and mask based on a network configuration file (see VxWorks Simulator Network Daemon Configuration and Use on page 40).

This section provides instructions for installing the optional simulator host connection driver. After a host driver is installed, vxsimnetd automatically configures its IP address and mask according to a default or custom configuration file. A packet sniffer such as **tcpdump**, **snoop**, or **Wireshark** can then be attached to the host interface to monitor traffic on the internal simulated subnet.

> Note: If you are working on a Windows host, installing the host connection driver (WRTAP driver), it can have an impact on your host system performance. Before installing this driver, review the information in Managing the WRTAP Driver on Windows on page 56.

2. Installing the Host Connection Driver on Windows

The same command is used to install the WRTAP driver on Microsoft Windows hosts, whether 32-bit or 64-bit.

About This Task

> Note: If you intend to use more than one external subnet, repeat the command for each subnet. You must install and configure a WRTAP driver for each subnet that is specified as external.

To install a Wind River WRTAP driver, proceed as follows:

Procedure

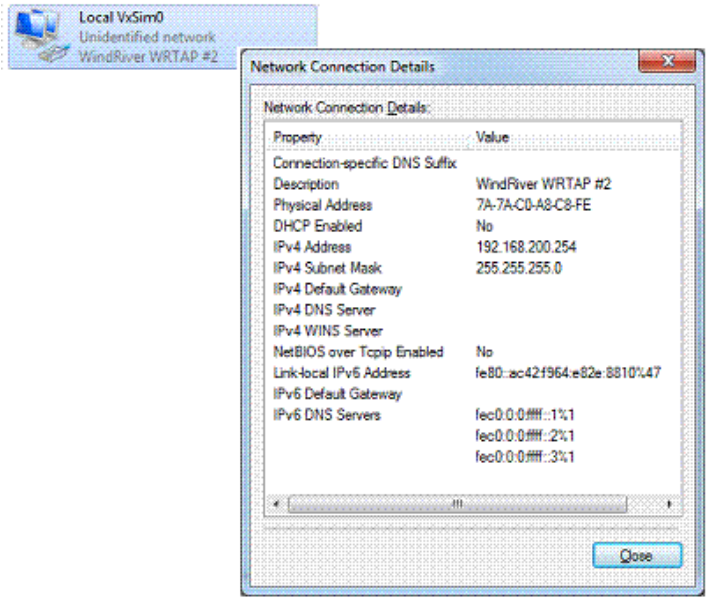
1. Open a Windows command shell as an administrator.
2. In the command shell, use the following command syntax:

   ```
   installDir\vxworks-7\host\x86-win32\bin\wrtapInstaller.exe install wrtap
   ```

   where *installDir* is the name of your VxWorks installation directory.

   You can view the new network connection present on your host from the network connections in the Windows Control Panel. The device type is Wind River WRTAP #*n*, where *n* can be 1, 2, 3, and so on. For example:

WNDRVR



3. Managing the WRTAP Driver on Windows

Managing the WRTAP is required to diagnose and correct any networking problems.

The following information may be useful when installing and using the host connection (WRTAP) driver on a Windows host. For instructions on installing the WRTAP driver, see Installing the Host Connection Driver on Windows on page 55.

Handling Networking Problems on Your Host System

If you encounter networking problems with the simulator or your host system after installing the WRTAP driver, you may need to make certain changes to your Windows network connection settings.

> Note: You need administrator privileges on the Windows host to make the following changes.

Certain communication protocols (particularly those which alter the maximum transmission unit (MTU) setting of the interface) can cause problems when the WRTAP driver is in use. In some instances, you may need to remove all protocols except TCP/IP. To do this, right click on each network connection and select **Properties**. Under the General tab, uncheck all items and components except TCP/IP.

> Note: Removing all protocols except TCP/IP has no impact on the general host system. The change only applies to the WRTAP interface that is used for communication between the host system and the simulator.

When you install the WRTAP driver, it becomes the primary network connection type on your host system. This can cause other applications to run slowly and can cause failures on the host system. To replace WRTAP as your main network connection, access the Advanced Settings dialog and under **Adapters and Bindings**, move your main Ethernet interface to the top of the **Connections** list.

Disabling the WRTAP Driver

IP address configuration for the WRTAP network connection is handled automatically by the VxWorks simulator network daemon (vxsimnetd). However, by default, the WRTAP network connection is turned on immediately following installation and uses DHCP to

configure its IP address. To avoid DHCP configuration, you can disable the WRTAP network connection after installation and allow it to be restarted and configured by the VxWorks simulator network daemon when necessary.

To disable the WRTAP network connection, access the Windows network connection settings by selecting **Start > Control Panel > Network Connections** (on Windows XP hosts) or **Start > Control Panel > Network and Sharing Center** (on Windows Vista hosts). In the Network Connections (or Network and Sharing Center) window, right-click the WRTAP network interface you want to disable and select Disable. (You can see that an interface is connected using the WRTAP driver by right-clicking and selecting Properties. The device the interface is using is listed in the **Connect using field** under the General tab.)

4. Installing the Host Connection Driver on Linux

You can install the host connection driver on Linux to set up a subnet that can communicate with your host.

To use TAP/TUN for vxsimnetd with a Linux host, you first need to create a TAP device. You can use either the tunctl or the openvpn command to install the host connection driver on Linux.

Installing the Host Connection Driver on Linux with tunctl on page 57 Use the tunctl command to install the host connection driver on Linux.
Installing the Host Connection Driver on Linux with openvpn on page 57 Use the openvpn command to install the host connection driver on Linux.

4.1. Installing the Host Connection Driver on Linux with tunctl

Use the tunctl command to install the host connection driver on Linux.

Procedure

1. Install tunctl if necessary:

```
$ sudo package_manager install tunctl
```

   where *package_manager* is a command such as apt-get, yum, or dnf, depending on your distribution.

2. Execute the tunctl command. For example:

```
$ sudo tunctl -t tap0 -u my_user_name
```

3. Start vxsimnetd.

   Once the TAP device is created, you can start vxsimnetd, but since configuring network interfaces on Linux is a privileged operation on most machines, running vxsimnetd in a privileged shell is usually required:

```
$ sudo ./wrenv.linux -p vxworks-7
$ vxsimnetd &
$ exit
```

Parent topic: Installing the Host Connection Driver on Linux on page 57

4.2. Installing the Host Connection Driver on Linux with openvpn

Use the openvpn command to install the host connection driver on Linux.

Procedure

1. Install openvpn if necessary:

```
$ sudo package_manager install openvpn
```

2. Create a tap device:

```
$ sudo openvpn --mktun --dev tap0
```

Parent topic: Installing the Host Connection Driver on Linux on page 57

8. NETWORKING ACROSS MULTIPLE HOSTS

1. Simulated Networks Across Multiple Hosts

Wind River provides the network daemon link utility wrnetdlink to link multiple simulated subnets running on multiple hosts.

The network daemon link utility (wrnetdlink) allows you to link multiple subnets running on a number of hosts by interconnecting the subnets from all hosts. wrnetdlink is required for large simulated environments where a single host does not have the necessary resources to run several networked simulator instances.

This utility enhances the functionality provided by the vxsimnetd tool.

The number of simulators that can be attached on a single host is limited by the processing power and the available memory of the host system. To ease this limitation, you can use multiple hosts and link their simulated subnets with wrnetdlink.

2. wrnetdlink Usage Scenarios

You use wrnetdlink from the host development shell once two or more hosts are configured with a simulated subnet.

Linking subnets on seperate hosts

To use the default configuration and link a subnet on the local host (host1) to a subnet on a second host (host2), use the following command syntax:

`wrnetdlink - net - net` *host2AddressOrName*

The default IP address (ipAddr) is 127.0.0.1 (local host). The default port number (portNumber) is 5555, which is the default value configured by **vxsimnetd**. The default subnet name is blank—if only one subnet on the host is available, it is used by default. Otherwise, the first subnet configured on the host is used.

Connecting Another Host to the Local Host

To connect the second host to the local host (host1), repeat the command on the second host. For example:

> **`wrnetdlink - net - net`** *host1AddressOrNamehost2AddressOrName*

Using Multiple Parameters

To specify each parameter for the wrnetdlink command, use the following command syntax:

`wrnetdlink - net` *ipAddr1* **:**_portNumber1_ **:**_subnetName1_ **`...`**
`- net` *ipAddrN* **:**_portNumberN_ **:**_subnetNameN_

ipAddrN

The IP address or host name for the host that is running the network daemon (each host you wish to link must be running a network daemon)

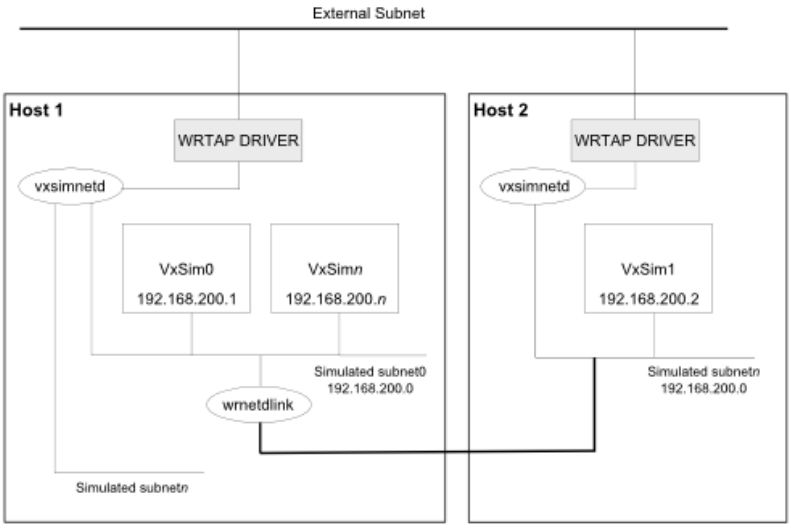
portNumberN

The port number that the network daemon uses to accept connections

subnetNameN

The name of the subnet running on the host

The diagram below illustrates the linking of subnets across two hosts. You can also start multiple wrnetdlink sessions to connect multiple multi-host subnets.



Getting Help

To get help for the wrnetdlink command, type wrnetdlink -help at the development shell prompt.

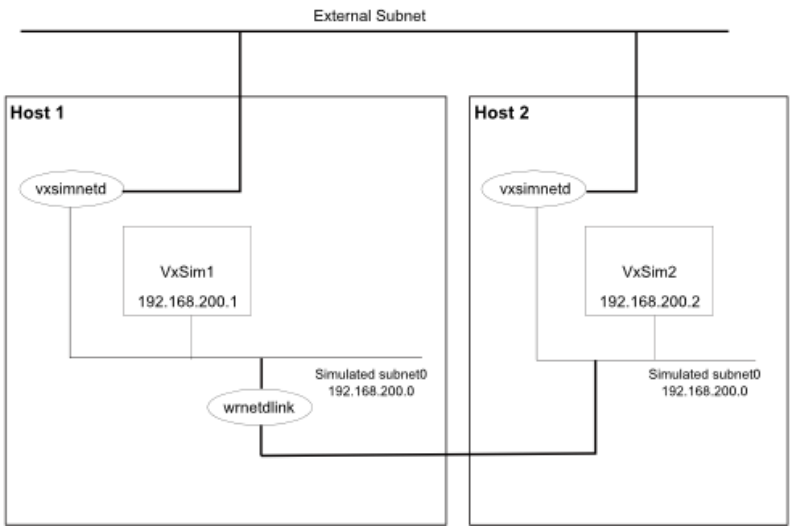
For an example implementation of wrnetdlink to link two simulated subnets across two hosts, see Networking Across Hosts with wrnetdlink Tutorial on page 60.

3. Networking Across Hosts with wrnetdlink Tutorial

This tutorial describes how to set up a simulated network across two hosts using the default vxsimnetd, vxsim, and wrnetdlink configuration.

You will begin by starting two simulated subnets, one on each of two separate host systems, and then link the simulated subnets using wrnetdlink.

The following diagram shows the network topology used in the tutorial.



To connect a subnet across two hosts:

1. Create a simulator-based VSB and VIP.
   See the Configuration and Build Workflow on page 4.
   To test connectivity, add the ping shell command component (**INCLUDE_IPPING_CMD**) to your VIP.
2. Install a host connection driver (WRTAP driver) and configure network connectivity to enable IP communications between the VxWorks simulator and your host.
   See Installing a WRTAP Driver for more details.
3. Start the VxWorks simulator network daemon vxsimnetd.
   See Launching the Network Daemon on Each Host on page 62.
4. Configure and start a VxWorks simulator instance with vxsim.
   See Starting a Simulator Instance on Each Host on page 63.
5. Connect the simulated subnets using wrnetdlink.
   See Connecting the Simulated Subnets Using wrnetdlink on page 64
6. Test the connection between the hosts.
   See Testing the Simulated Network on page 65.

Note:

This tutorial makes a connection between two host systems but any number of host systems can be connected—or chained together—in a similar fashion.

The host systems can be of any supported host type. For example, you can connect a Windows host to a Linux host.

Installing a WRTAP Driver on page 62 To simulate network IP connectivity to your host operating system, install a WRTAP driver.
Launching the Network Daemon on Each Host on page 62 You must launch the network daemon (vxsimnetd) on each host before connecting them.
Starting a Simulator Instance on Each Host on page 63 You must start a simulator on each host before subnets can be connected.
Connecting the Simulated Subnets Using wrnetdlink on page 64 Use wrnetdlink to connect simulated subnets on multiple hosts.
Testing the Simulated Network on page 65 Use the ping command to test connectivity between the simulated subnets.

WNDRVR

3.1. Installing a WRTAP Driver

To simulate network IP connectivity to your host operating system, install a WRTAP driver.

About This Task

Once installed, the WRTAP driver is automatically initialized upon launching vxsim.

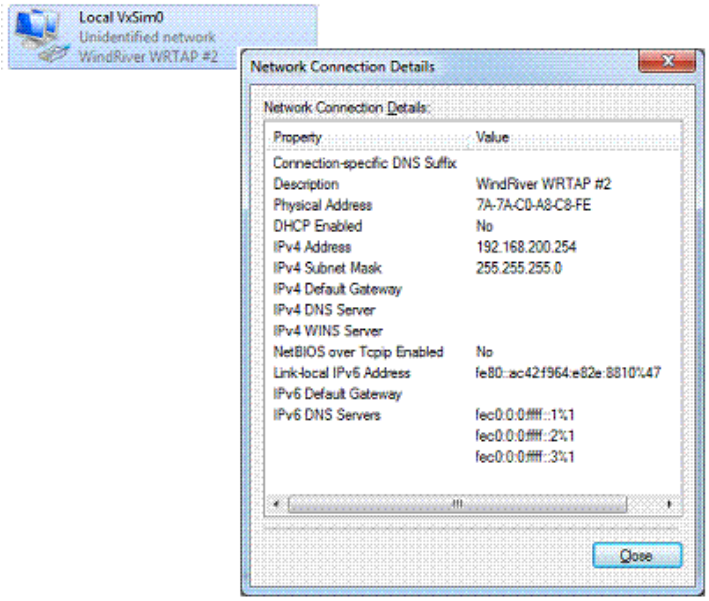
Procedure

1. Open a Windows command shell as an administrator.
2. In the command shell, use the following command syntax:

```
installDir\vxworks-7\host\x86-win32\bin\wrtapInstaller.exe install wrtap
```

where *installDir* is the name of your VxWorks installation directory.

You can view the new network connection present on your host from the network connections in the Windows Control Panel. The device type is Wind River WRTAP #*n*, where *n* can be 1, 2, 3, and so on. For example:



Parent topic: Full Network Simulation Using simnet on page 27

Parent topic: Networking Across Hosts with wrnetdlink Tutorial on page 60

3.2. Launching the Network Daemon on Each Host

You must launch the network daemon (vxsimnetd) on each host before connecting them.

Getting Help

To get help for vxsimnetd, use the vxsimnetd -help command from the development shell.

For more information on vxsimnetd, see VxWorks Simulator Network Daemon Configuration and Use on page 40.

About This Task

To set up the daemon on each host using the default configuration file (*installDir* **\vxworks-7\host\resource\vxsim\vxsimnetd.conf**), do the following:

Procedure

1. Open a Windows command shell as an administrator.
2. Launch the simulator using the default configuration. Use the following command:

```
> vxsimnetd -s
```

3. Display the current network configuration and status of the default subnet with the following commands:

```
vxsimnetd> subnet default
vxsimnetd> node default
vxsimnetd> packet default
```

   The default class C subnet is at 192.168.200.0 and has 256 IP addresses.

4. Repeat steps 1 to 3 for the second host.

Parent topic: Networking Across Hosts with wrnetdlink Tutorial on page 60

3.3. Starting a Simulator Instance on Each Host

You must start a simulator on each host before subnets can be connected.

Getting Help

To get help for the wrenv command, enter wrenv -help at the development shell prompt.

To get help for the vxsim command, enter vxsim -help at the development shell prompt.

For more information on vxsim, see VxWorks Simulator Options Reference on page 14 .

Procedure

1. Open a development shell either from Workbench 4 or from the command line.

   From the command line, use the following command:

```
installDir>wrenv -p vxworks-7
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

installDir>
```

2. Access the default directory of the VIP. For example:

WNDRVR

```
/installDir /workspace/my_vip /default>
```

3. On the first host, launch the simulator instance (VxSim1) from the VxWorks development shell as follows:

```
>vxsim -d simnet -e 192.168.200.1 -p 1
```

4. On the second host, launch the simulator instance (VxSim2) from the VxWorks development shell as follows:

```
>vxsim -d simnet -e 192.168.200.2 -p 2
```

5. View the network configuration for the targets.

   Use the ifconfig command from the kernel shell of each target.

   For example:

```
-> ifconfig
ifconfig
...
simnet0Link type:Ethernet HWaddr 7a:7a:c0:a8:c8:01
    inet 192.168.200.1 mask 255.255.255.0 broadcast 192.168.200.255
    inet6 unicast fe80::787a:c0ff:fea8:c801%simnet0 prefixlen 64 automatic
    UP RUNNING SIMPLEX BROADCAST MULTICAST
    MTU:1500 metric:1 VR:0 ifindex:2
    RX packets:56 mcast:0 errors:0 dropped:0
    TX packets:71 mcast:8 errors:0
    collisions:0 unsupported proto:0
    RX bytes:2640 TX bytes:11k
value = 0 = 0x0
->
```

Parent topic: Networking Across Hosts with wrnetdlink Tutorial on page 60

3.4. Connecting the Simulated Subnets Using wrnetdlink

Use wrnetdlink to connect simulated subnets on multiple hosts.

About This Task

Procedure

From the VxWorks development shell on the first host, launch the wrnetdlink utility using the following command syntax:

```
>wrnetdlink -net -net host2
```

where *host2* is the IP address or host name of the second host.

The first -net flag (with no parameters) specifies a connection to the local host network daemon on port 5555 and connecting to the first configured subnet (in this case, the default is 192.168.200.0). The second -net flag specifies a connection to the network daemon running on host2 using the default port and connecting to the first configured subnet.

Parent topic: Networking Across Hosts with wrnetdlink Tutorial on page 60

WNDRVR

3.5. Testing the Simulated Network

Use the ping command to test connectivity between the simulated subnets.

About This Task

You can now test the simulated network using the ping command.

Getting Help

To get help for the ping command, type ping at the command shell prompt.

Procedure

1. Ping the simulator instance on the first host from the simulator instance on the second host:

```
# ping 192.168.200.1
```

2. Ping the simulator instance on the second host from the simulator instance on the first host:

```
# ping 192.168.200.2
```

Parent topic: Networking Across Hosts with wrnetdlink Tutorial on page 60

9. API FUNCTIONS

1. VxWorks Simulator API Library (vxsimapi) Reference

The VxSim **vxsimapi** API library contains API functions available to VxSim applications.

Function	Description
vxsimCpuIndexGet( )	Retrieve the current VxSim CPU index.
vxsimExitHookAdd( )	Specify a function to be called upon simulator exit or reboot.
vxsimFdIntDisable( )	Disable file descriptor interrupts (Linux).
vxsimFdIntEnable( )	Enable file descriptor interrupts (Linux).
vxsimIntAckRtnAdd( )	Add an acknowledge function for a given interrupt vector (Windows).
vxsimIntRaise( )	Raise an interrupt to VxWorks (Windows).
vxsimIntToMsg( )	Convert a VxWorks interrupt vector to a windows message (Windows).
vxsimMsgToInt( )	Convert a Windows message to a VxWorks interrupt vector (Windows).
vxsimWindowHandleGet( )	Retrieve the current VxSim top-level window handle (Windows).

2. Windows Serial Communications API Library (commSio) Reference

The **commSio** library contains Windows-specific API functions for the hostSio driver provided with the **vxsim_windows** BSP.

The library module is provided as a simulator host driver. The commSio API functions are only accessible by the hostSio driver functions. For information on how to build, load, and use the library functions, see the reference entry for **commSio**.

Function	Description
commSioBaudRateSet( )	Change the device baud rate.
commSioClose( )	Close a serial device.
commSioDataGet( )	Read data from a serial device.
commSioDataSend( )	Send data to a serial device.

Function	Description
commSioInit( )	Initialize the Windows **commSio** library.
commSioIntClear( )	Disable the interrupt mode for a serial device.
commSioIntSet( )	Set the interrupt mode for a serial device.
commSioOpen( )	Open a serial device.

3. Linux Serial Communications API Functions (ttySio) Reference

The **ttySio** library contains Linux-specific API functions for the hostSio driver provided with the **vxsim_linux** BSP.

The library module is provided as a simulator host driver. The **ttySio** API functions are only accessible by the hostSio driver functions. For information on how to build, load, and use the library functions, see the reference entry for **ttySio**.

Function	Description
ttySioBaudRateSet( )	Change the device baud rate.
ttySioClose( )	Close the serial device.
ttySioDataGet( )	Read data from the serial device.
ttySioDataSend( )	Send data to the serial device.
ttySioInit( )	Initialize the Linux **ttySio** library.
ttySioIntClear( )	Disable interrupt mode for serial devices.
ttySioIntSet( )	Set the serial device in interrupt mode.
ttySioOpen( )	Open a serial device.

4. VxWorks Simulator Host Interface Library (vxsimHostArchLib) Reference

The VxSim **vxsimHostArchLib** host interface library contains API functions available to VxSim applications.

Function	Description
vxsimHostDllLoad( )	Load a dynamic link library (DLL) into the simulator.
vxsimHostProcAddrGet( )	Return the address of a host API function.
vxsimHostProcCall( )	Call a host API function.

WNDRVR

Function	Description
vxsimHostExcEnterHook( )	Specify a function to be called upon an software exception.

Note:

If you are using the VxSim to simulate multiple processors in SMP mode, you should be cautious when calling host functions from the VxSim code that uses vxsimHostProcAddrGet( ) to make sure it is SMP-safe. Note that vxsimHostProcCall( ) replaces a direct function pointer dereference that is forbidden in SMP. For more information see Accessing Host OS Functions on page 85.

10. VXWORKS SIMULATOR ENVIRONMENT

1. Introduction

The VxSim environment differs very little from the development environment for any other target hardware system

The most notable differences are the networking methods, which are discussed in VxWorks Simulator Network Daemon Configuration and Use on page 40, and variations in the simulator BSP and architecture behavior, which are discussed in the following sections.

For information on BSPs, see the *VxWorks 7 BSP and Drivers Guide*.

Wind River highly recommends that you build simulator modules using the wrtool command-line facility or Wind River Workbench; however, if you wish to customize your build, you may need the information in the following sections.

2. Compiler Options

The build mechanism for the VxSim uses two preprocessor constants, CPU and TOOL, to define compiler options for a specific build target.

The CPU variable ensures that VxWorks and your applications are compiled with the appropriate features enabled. The TOOL variable defines the toolchain to use for compiling and linking modules.

Note: The VxWorks simulator image is built with the Clang (LLVM) compiler (no other compilers are supported for this purpose). RTP and kernel applications are built with the Clang (LLVM) compiler by default. For information about developing applications, and about C++ support, see *VxWorks 7 C++ Programmer's Guide: C++ Support Overview*. To configure compiler options for your VSB, see VSB Configuration Reference on page 4.

To build modules using the Clang (LLVM) compiler, define TOOL as llvm.

Applications for the simulator can be built to run in either the VxWorks kernel or a VxWorks RTP.

The following table lists the available CPU and TOOL definitions for the simulator. It also provides sample command-line options specific to the simulator architecture. For more information on the available compiler options, see the compiler documentation for Pentium (Windows and Linux hosts).

CPU	Run Type	Tool	Compiler Command-Line Options
SIMNT	Kernel and RTP	llvm	-DCPU=SIMNT
SIMLINUX	Kernel and RTP	llvm	-DCPU=SIMLINUX
SIMNT	64-bit Kernel and RTP	llvm	-m64 -mcmodel=large -mno-red-zone -fno-implicit-float -fno-omit-frame-pointer -DCPU=SIMNT
SIMLINUX	64-bit Kernel and RTP	llvm	-m64 -mcmodel=large -mno-red-zone -fno-implicit-float -fno-omit-frame-pointer

CPU	Run Type	Tool	Compiler Command-Line Options
			-DCPU=SIMLINUX

On all hosts, the simulator uses ELF object module format (OMF). Your VxWorks installation also includes a simulator binary for each supported host system. You can use this binary as a boot loader for a VxWorks image that you can configure and build using exactly the same compiler options you use to build a VxWorks image for a hardware target architecture. Thus, if you are compiling a VxWorks image for a Linux or Windows simulator, you should use the same compiler as the Intel Architecture.

If you attempt to load an ELF image for another architecture onto a simulator, you will get a load error. For example, if you attempt to load a PPC32 ELF image from the kernel shell:

```
% ld < host:C:/WindRiver/workspace/PPC_ELF.out
ld(): error loading file (errno = 0x610001).
value = 0 = 0x0
```

For information on available compiler options, see http://clang.llvm.org/ .

3. Compiling Modules for Debugging

To compile C and C++ modules for debugging, you must use the -g flag to generate debug information.

About This Task

Example command-line sequences for the CLANG (LLVM) compiler are as follows:

Procedure

Enter one of the following to enable debugging support. In the following examples, *installDir* is the location of the VxWorks installation tree.

- In the following Intel example, -DCPU specifies the VxSim on the Linux host.

```
% clang --target=i386 -march=i486 -IinstallDir/workbench-4/workspace/cpp_demo -DCPU=_VX_SIMLIN
UX -DTOOL_FAMILY=llvm -DTOOL=llvm -g test.cpp
```

- The following is the equivalent 64-bit Intel example. The parameters --target=x86_64 and -m64 signal that this is a 64-bit compilation.

```
% clang --target=x86_64 -m64 -mcmodel=large -mno-red-zone -fno-implicit-float -fno-omit-frame-
pointer -IinstallDir/workbench-4/workspace/cpp_demo -DCPU=_VX_SIMLINUX -DTOOL_FAMILY=llvm -DTO
OL=llvm -g test.cpp
```

Note:

Debugging code compiled with optimization is likely to produce unexpected behavior, such as breakpoints that are never hit or an inability to set breakpoints at some locations. This is because the compiler may re-order instructions, expand loops, replace functions with in-line code, and perform other code modifications during optimization, making it difficult to correlate a given source line to a particular point in the object code. Users are advised to be aware of these possibilities when attempting to debug optimized code. Alternatively, users may choose to debug applications without using compiler optimization.

To compile without optimization using the Clang (LLVM) compiler, compile with the -O0 option.

4. Interface Variations

The VxSim uses the VxWorks MMU and file system support tools differently than the standard VxWorks implementation.

This section describes particular functions and tools that are specific to simulator targets in any of the following ways:

- Available only for simulator targets
- Parameters specific to simulator targets
- Special restrictions on, or characteristics of, simulator targets

For complete documentation, see the reference entries for the libraries, functions, and tools discussed in the following sections.

Memory Management Unit

The memory management unit implementation for the VxSim differs from the standard VxWorks MMU implementation.

Simulation

The simulator provides a simulated hardware memory management unit. The simulated MMU provides features comparable to those found on typical hardware MMUs. The simulation uses features provided by the host operating system to map, unmap, and protect pages in memory. MMU simulation is provided for all supported host operating systems.

Page Size

The simulator uses a page size that is determined by the memory page mapping functions in the host operating system. On Linux-based simulators, this page size is 8 KB. On Windows simulators, this page size is 64 KB.

Limitations

The simulator MMU implementation does not provide support for supervisor/user mode.

RTP Considerations

Because the simulator MMU implementation does not support supervisor/user mode, it is not possible to prevent a task running in an RTP from writing in the kernel memory space. Therefore, on the simulator architecture, an RTP task can potentially crash a kernel task.

File System Support

The simulator MMU implementation does not provide support for supervisor/user mode.

Pass-Through File System (passFS)

By default, the simulator uses a pass-through file system (passFS) to access files directly on the host system. For more information on using passFS with the simulator, see the discussion of the *Pass-Through File System (passFS)* in Configuring Hardware Simulation on page 81

Virtual Disk Support

The simulator provides virtual disk support which allows you to simulate a disk block device. The simulated disk block device can be used to access any file system supported by VxWorks. For more information on virtual disk support for the simulator, see the discussion of *Virtual Disk Support* in Configuring Hardware Simulation on page 81.

WNDRVR

5. Architecture Considerations

The memory management unit implementation for the VxSim varies from the standard VxWorks MMU implementation.

The following topics are addressed:

- byte order
- hardware breakpoint
- floating-point support
- ISR stack protection
- interrupts
- memory

Byte Order

The Windows and Linux simulators use a little-endian storage method.

Hardware Breakpoint

The simulator does not support hardware breakpoints.

ISR Stack Protection

ISR stack overflow and underflow protection is supported on Linux simulators. The simulator does not require ISR stack overflow and underflow protection on Windows simulators because the Windows operating system automatically detects this type of error condition and handles it before VxWorks can take action.

For more information on ISR stack protection, see the *VxWorks 7 Platform Programmer's Guide:Interrupt Service Routines: ISRs*.

Floating-Point Support Reference on page 72 For 32-bit VxWorks, Windows and Linux simulators use i387 floating-point registers and instructions. For 64-bit VxWorks, these simulators use SSE instructions.
Interrupts on page 73 The simulator environment provides conventions for interrupt handling on both Linux and Windows.
Memory Configuration on page 76

5.1. Floating-Point Support Reference

For 32-bit VxWorks, Windows and Linux simulators use i387 floating-point registers and instructions. For 64-bit VxWorks, these simulators use SSE instructions.

acos( )	**asin( )**	**atan( )**	**atan2( )**
cos( )	**cosh( )**	**exp( )**	**fabs( )**
floor( )	**fmod( )**	**log( )**	**log10( )**
pow( )	**sin( )**	**sinh( )**	**sqrt( )**
tan( )	**tanh( )**		

The following floating-point functions are not available on the simulator:

cbrt( )	**ceil( )**	**infinity( )**	**irint( )**
iround( )	**log2( )**	**round( )**	**sincos( )**
trunc( )	**cbrtf( )**	**infinityf( )**	**irintf( )**
iroundf( )	**log2f( )**	**roundf( )**	**sincosf( )**
truncf( )			

In addition, the following single-precision functions are not available:

acosf( )	**asinf( )**	**atanf( )**	**atan2f( )**
ceilf( )	**cosf( )**	**expf( )**	**fabsf( )**
floorf( )	**fmodf( )**	**logf( )**	**log10f( )**
powf( )	**sinh( )**	**sinhf( )**	**sqrtf( )**
tanf( )	**tanhf( )**		

Parent topic: Architecture Considerations on page 72

5.2. Interrupts

The simulator environment provides conventions for interrupt handling on both Linux and Windows.

Interrupts on Linux Hosts

On Linux simulators, the hardware interrupt simulation is performed using host signals. For example, the simulator uses the **SIGALRM** signal to simulate system clock interrupts.

Furthermore, all host file descriptors (such as standard input) can be put in asynchronous mode, so that the **SIGPOLL** signal is sent to the simulator when data becomes available. For more information on how to configure a host device to generate interrupts when data is available, see Host Functions and Code Access on page 85.

For the simulator, signal handlers provide the equivalent functionality of interrupts available on other target architectures. You can install ISRs in the simulator to handle these interrupts.

> Note: Not all VxWorks functions can be called from ISRs. For more information, see the *VxWorks 7 Platform Programmer's Guide:Interrupt Service Routines: ISRs*.

To run ISR code during a future system clock interrupt, use the watchdog timer facilities. To run ISR code during auxiliary clock interrupts, use the sysAuxClk( ) functions.

The following table shows how the Linux simulator interrupt vector tables are assigned.

Interrupt Vectors	Description
1...	Host signal 1
SIGUSR1SIGUSR2...	User signal 1User signal 2
32	Host signal 32
33...	Interrupt vector for host file descriptor 1 (SIGPOLL)
288	Interrupt vector for host file descriptor 256 (SIGPOLL)

When you write your VxBus driver, you would usually connect the interrupt in the attach( ) function with the following code:

```
#include <arch/simlinux/ivSimlinux.h>
...
pDrvCtrl->pRes = (VXB_RESOURCE *)vxbMemAlloc (sizeof (VXB_RESOURCE));
if (pDrvCtrl->pRes != NULL)
    {
    pResIrq = pDrvCtrl->pRes->pRes =
        (VXB_RESOURCE_IRQ *)vxbMemAlloc (sizeof(VXB_RESOURCE_IRQ));
    if (pResIrq != NULL)
       {
       pResIrq->lVec = IV_USR2;
       pResIrq->hVec = IV_USR2;
       pResIrq->flag |= VXB_INT_FLAG_STATIC;
       }
    pDrvCtrl->pRes->id = VXB_RES_ID_CREATE(VXB_RES_IRQ, 0);
    }
(void)vxbIntConnect (pDrvCtrl->pDev, pDrvCtrl->pRes,
       handler, (void *)pDrvCtrl);
```

Note: In your VxWorks applications, avoid using the preprocessor constants **SIGUSR1** or **SIGUSR2**. VxWorks defines its own values for these constants and those values differ from the host definitions. Therefore, you must specify the host signal numbers explicitly in your VxWorks application code.

Only **SIGUSR1** and **SIGUSR2** can be used to represent user-defined interrupts (see the following table).

Signal	Linux Value
SIGUSR1	10
SIGUSR2	12

Interrupts on Windows Hosts

On Windows, the simulator uses Windows messages to simulate hardware interrupts. For example, the simulator uses messages to simulate interrupts from the network connections, the pipe back end, and so forth.

For the simulator, messages provide the equivalent functionality of interrupts available on other target architectures. You can install ISRs in the simulator to handle these messages.

WNDRVR

Note: Not all VxWorks functions can be called from ISRs. For more information, see the *VxWorks 7 Platform Programmer's Guide:Interrupt Service Routines: ISRs*.

To run ISR code during a future system clock interrupt, use the watchdog timer facilities. To run ISR code during auxiliary clock interrupts, use the sysAuxClk( ) functions.

The following table shows how the Windows simulator interrupt vector table is assigned.

Interrupt Vectors	Description
0x0000	System clock interrupt.
0x0001	Auxiliary clock interrupt.
0x0002	Timestamp rollover interrupt.
0x0003	Back-end pipe interrupt.
0x0004	SIO driver interrupt.
0x0005	Bus interrupt.
0x0006-0x0009	Inter-processor interrupts (IPIs).
0x0009-0x00ef	Reserved for internal use.
0x00f0-0x00ff	Wind River Media Library interrupt range.
0x180-0x01ff	Simulated network interrupt range.
0x0200-0x02ff	User interrupt range.

When you write your VxBus driver, you would usually connect the interrupt in the attach( ) function with the following code:

```
#include <arch/simlinux/ivSimnt.h>
...
pDrvCtrl->pRes = (VXB_RESOURCE *)vxbMemAlloc (sizeof (VXB_RESOURCE));
if (pDrvCtrl->pRes != NULL)
    {
    pResIrq = pDrvCtrl->pRes->pRes =
        (VXB_RESOURCE_IRQ *)vxbMemAlloc (sizeof(VXB_RESOURCE_IRQ));
    if (pResIrq != NULL)
        {
        pResIrq->lVec = TIMER_AUX_CLK_INT;
        pResIrq->hVec = TIMER_AUX_CLK_INT;
        pResIrq->flag |= VXB_INT_FLAG_STATIC;
        }
    pDrvCtrl->pRes->id = VXB_RES_ID_CREATE(VXB_RES_IRQ, 0);
    }
(void)vxbIntConnect (pDrvCtrl->pDev, pDrvCtrl->pRes,
    handler, (void *)pDrvCtrl);
```


WNDRVR

The user interrupt range can be used by the host side user application. For more information on using user interrupts, see Host Functions and Code Access on page 85.

Parent topic: Architecture Considerations on page 72

5.3. Memory Configuration

32-Bit Simulator Memory Layout

The VxWorks memory layout is the same for all 32-bit VxWorks simulators. The following diagram shows the memory layout, labeled as follows:

Boot Line

ASCII string of boot parameters.

Exception Message

ASCII string of the fatal exception message.

System Image

The VxWorks system image itself (three sections: text, data, and bss). The entry point for VxWorks is at the start of this region.

Kernel Virtual Memory Pool Region

The kernel virtual memory pool region is the area used for dynamically managed mappings in the kernel. This region is used to allocate virtual memory as needed for creation and expansion of the kernel application, user-reserved memory and persistent memory.

User Reserved Memory

Application-reserved memory. Its size is configured through the USER_RESERVED_MEM parameter. By default, this memory region is cleared upon reboot; however, it can be changed using the CLEAR_USER_RESERVED_MEMORY_ON_COLD_BOOT parameter.

ED&R Persistent Memory

A reboot-safe protected memory region manager.

Shared User Virtual Memory Region

This is the shared memory region that is used for creating the shared virtual memory pool. It is used to allocate code and data segments of shared libraries.

RTP Private Virtual Memory Region

This region is added to each RTP's page manager upon creation. It is used for allocating the RTP's code segments and private mappings.

Kernel System Virtual Memory Region

This region includes the following elements:

- boot line
- exception message
- system image

Interrupt Stack

Size is defined by ISR_STACK_SIZE under **INCLUDE_KERNEL**. The location depends on the system image size.

Interrupt Vector Table

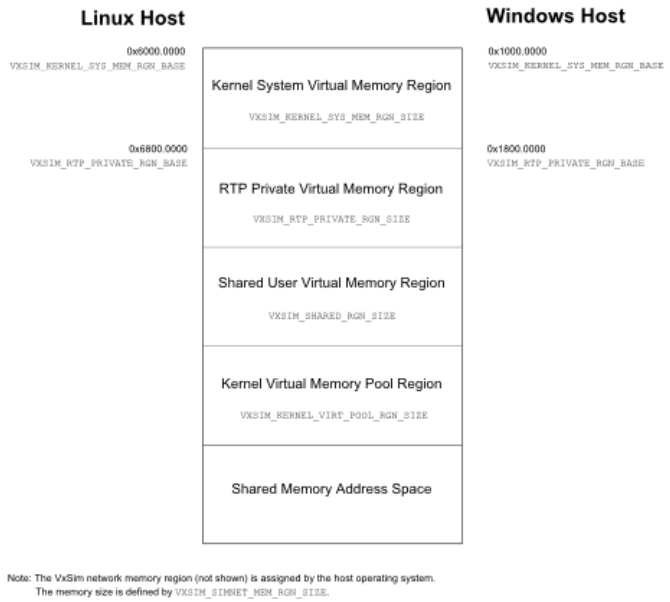
Table of interrupt vectors.

Shared Memory Address Space

Address space reserved for user shared memory (if included).

Networking Address Space

Address space for simulator full networking (simnet), if network support is included.



64-Bit Simulator Memory Layout

The following diagram shows the memory layout for the 64-bit simulator on 64-bit Windows and Linux hosts.

Kernel Virtual Memory Pool Region

The kernel virtual memory pool region is the area used for dynamically managed mappings in the kernel. This region is used to allocate virtual memory as needed for creation and expansion of the kernel application, user-reserved memory and persistent memory.

Shared User Virtual Memory Region

This is the shared memory region that is used for creating the shared virtual memory pool. It is used to allocate code and data segments of shared libraries.

RTP Private Virtual Memory Region

This region is added to each RTP's page manager upon creation. It is used for allocating the RTP's code segments and private mappings.

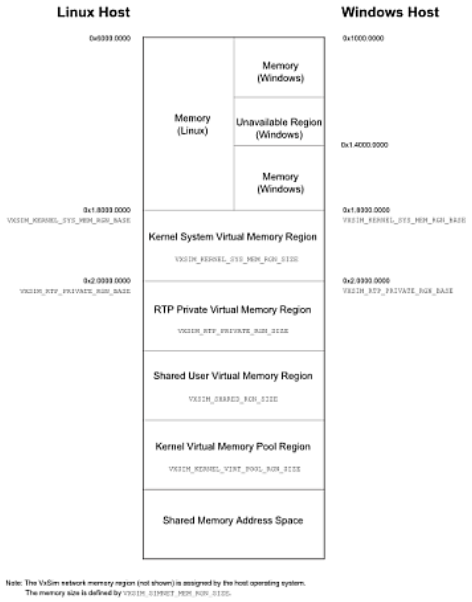
Kernel System Virtual Memory Region

This region includes the following elements:

- boot line
- exception message

WNDRVR

- system image



Memory Configuration at Startup

The simulator displays its memory settings at startup, as shown in the following Windows host examples.

32-Bit VxWorks Example

```
Virtual Base Address: 0x10000000
Virtual Top Address: 0x48000000 Virtual Size: 0x38000000 (896Mb)
Physical Base Address: 0x10000000
Physical Top Address: 0x30000000 Physical Size: 0x20000000 (512Mb)
```

64-Bit VxWorks Example

```
Virtual Base Address: 0x10000000
Virtual Top Address: 0x48000000 Virtual Size: 0x38000000 (896Mb)
Physical Base Address: 0x10000000
Physical Top Address: 0x30000000 Physical Size: 0x20000000 (512Mb)
```

Physical Memory Address Space

The simulator physical memory address space is defined by the **LOCAL_MEM_LOCAL_ADRS** and **LOCAL_MEM_SIZE** parameters in the simulator BSP. The simulator uses the values that were set for these parameters when you built VxWorks.

However, the simulator physical memory size can be dynamically modified with vxsim -size for **LOCAL_MEM_LOCAL_ADRS** or vxsim -memsize for **LOCAL_MEM_SIZE**.

Note: The **LOCAL_MEM_ADRS** parameter must be aligned to 1 MB (0x100000) and the **LOCAL_MEM_SIZE** parameter must be a multiple of 1 MB.

WNDRVR

Note: Memory is added to the kernel virtual pool instead of the kernel system virtual memory region and is thus not visible with memShow( ).

Virtual Memory Address Space (32-Bit VxWorks Simulator)

For the 32-bit simulator, the virtual memory size is limited to 1 GB.

On Windows hosts, the simulator virtual base address is 0x10000000 and the simulator virtual top address is 0x4FFFFFFF.

On Linux hosts, the simulator virtual base address is 0x60000000 and the simulator virtual top address is 0x9FFFFFFF.

You can modify memory size using parameters shown in the diagram *32-Bit VxWorks System Memory Layout (VxWorks Simulator)*, above.

Virtual Memory Address Space (64-Bit VxWorks Simulator)

For the 64-bit simulator, the virtual memory size is 14 GB by default.

On Windows hosts, the simulator virtual base address is 0x10000000.

On Linux hosts, the simulator virtual base address is 0x60000000.

You can modify memory size using parameters shown in the diagram *64-Bit VxWorks System Memory Layout (VxWorks Simulator)*, above.

User Shared Memory - Creating a User Shared Memory Area

Shared memory is available for your own implementations of inter-process communication.

To set up this kind of shared memory in your simulator instance, include the **INCLUDE_VXSIM_USER_SM** configuration component in your VxWorks image.

The size of the user shared memory area is configurable. When you include the **INCLUDE_VXSIM_USER_SM** component, use its **VXSIM_USER_SM_SIZE** parameter to specify the size. The size value is rounded up to the MMU page size.

User Shared Memory - Multiple Simulator Instances

The user shared memory area can be shared by one or more simulator instances.

Warning: All simulator instances that are connected to shared memory must have the same shared memory configuration, including for the user shared memory area. The size of the user shared memory area must be the same for all simulator instances that use it. If you use different shared memory configurations for different instances, unpredictable results may occur.

User Shared Memory - Getting Information on the User Shared Memory Area

Once you have built the simulator image and started it, you can view the current size and base address of the area through the sysUserSmInfoGet( ) function.

If user shared memory is not configured in the system, this function returns 0.

WNDRVR

Memory Protection Level

The simulator allows you to specify a memory protection level using the -prot-level option. This level can be set to min (0) to provide no protection or max (1) where stack overflow protection is provided. By default, the memory protection is set to the maximum level (max).

Parent topic: Architecture Considerations on page 72

6. Boot Parameters

Using the command-line interface, you can specify all parameters available in the standard VxWorks boot line for a simulator target.

However, you lose these parameters when you exit the simulator even if you include non-volatile RAM support. This occurs because, even when the specified boot parameters are saved in a file (**nvram.***vxWorkscpuNum*) similar to actual hardware target behavior, they are only used for a target reboot, not when the target is exited or restarted.

Once the simulator starts, you can use the bootChange and set bootline commands to modify boot line parameters. The new parameters are preserved and taken into account on the next reboot.

To view the current boot parameters, enter the command interpreter mode and use the show bootline command. For example:

```
-> cmd
[vxWorks *]#show bootline
boot device : passDev
unit number : 0
processor number : 0
host name : host
file name :
C:/vx7git/vxworks/workbench-4/workspace/vip_vsb_vx7_vxsim/default/vxWorks
user (u) : user
flags (f) : 0x0
target name (tn) : VxSim0
```

To change and save the boot parameters for future sessions, use the set bootline or bootChange command. For example:

```
[vxWorks *]#set bootline
[vxWorks *]#bootChange
```

or

```
[vxWorks *]#C
-> bootChange
'.' = clear field; '-' = go to previous field; ^D = quit
boot device : passDev0
processor number : 0
host name : host MyHost0
file name : ^D
value = 0 = 0x0
```

To reboot the instance and view the change to the host name field, use the following commands.

WNDRVR

```
-> reboot
...
-> cmd
[vxWorks *]# show bootline
boot device : passDev
unit number : 0
processor number : 0
host name : MyHost0
file name :
C:/vx7git/vxworks/workbench-4/workspace/vip_vsb_vx7_vxsim/default/vxWorks
user (u) : user
flags (f) : 0x0
target name (tn) : VxSim0
```

7. Configuring Hardware Simulation

The VxSim provides hardware simulation options for your development environment.

Non-Volatile RAM Support

By default, a VxWorks image includes support for non-volatile RAM, which has a default size of 256 bytes. This memory is dedicated to storing boot line information. To store anything else in NVRAM, use the **NV_RAM_SIZE** macro to increase the size of NVRAM. To access NVRAM, use sysNvRamSet( ) and sysNvRamGet( ).

To simulate NVRAM, the simulator uses a file on the host system. By default, this file resides in the same directory that contains the VxWorks image. To specify another location, use the -nvram command-line option:

vxsim -nvram *pathToFileForNVRAM*

Pass-Through File System (passFS)

The default file system for the simulator is the pass-through file system (passFS). This file system is unique to the simulator. The **INCLUDE_PASSFS** component is included by default and mounts this file system on startup. passFS is a file-oriented device driver that provides easy access to the host file system. To specify the passFS device name (the default is your system host name), use the following command-line option:

vxsim -hn *hostname*

or

vxsim -hostname *hostname*

On Linux hosts, the default value for the passFS device name is the name of the host on which the simulator is running. On Windows, for backward compatibility with previous releases, the default value is host.

The VxWorks syntax for accessing a host file system is summarized in the table below.

WNDRVR

Table 1. VxWorks Syntax for Accessing passFS

Host Type	passFS Syntax	Example
Linux	`passFSDevice:/dir/file`	`ls myhost:/myDir/myFile` (where host name is myHost)
Windows	`passFSDevice:/disk/dir/file`	`ls host:/c/myDir/myFile`
Windows (deprecated; syntax preserved for backward compatibility)	`passFSDevice:disk:/dir/file`	`ls host:c:/myDir/myFile`

Note: passFS uses UNIX-style path separators (/) even on a Windows-based simulator.

Serial Host Line Support

The simulator provides a sample host serial I/O driver (hostSio). This driver provides access to a host serial device from the simulator. This feature is not included in the default simulator. To add host serial device support, the **INCLUDE_HOST_SIO** component must be defined in the BSP configuration. The parameter **HOST_SIO_PORT_NUMBER** can be used to select which host serial device to use. For more information, see Host Functions and Code Access on page 85.

Serial Input and Output (I/O)

The simulator BSPs provide a serial input and output (SIO) driver to handle serial input and output. The driver is at *installDir* **/ vxworks-7/pkgs_v2/os/arch/simulator/kernel/src/drv/vxbVxsimSio**

Note: On Linux simulators, UNIX job control characters are enabled even when the I/O is in raw mode. Trapping of control characters like **^Z** is UNIX-shell specific and does not conform to the usual VxWorks **tyLib** conventions. Trapping of the **^C** character is performed by the kernel shell (when it is included in your image).

Timers

Similar to any VxWorks target, the simulator provides a system clock and an auxiliary clock. The macros **SYS_CLK_RATE_MIN**, **SYS_CLK_RATE_MAX**, **AUX_CLK_RATE_MIN**, and **AUX_CLK_RATE_MAX** are defined to provide parameter checking for the sysClkRateSet( ) and sysAuxClkRateSet( ) functions.

Note: If the simulator process is preempted by another process on the host machine, the simulator clock can be impacted. In such cases, the current activity of each VxWorks task is delayed by an interval of time that corresponds to the preempted time of the process.

Timestamp Driver

The simulator provides a system-defined timestamp driver. In general, this driver is used to extend the range of information available from VxWorks kernel instrumentation. For example, when a timestamp driver is available, a precise time line can be displayed using the Wind River System Viewer.

The timestamp driver is included in the default simulator configuration. The timestamp driver is selected by including **INCLUDE_TIMESTAMP** (32-bit) or **INCLUDE_TIMESTAMP64** (64-bit) in your VxWorks image

WNDRVR

Virtual Disk Support

To simulate access to file systems, either the file system supplied with VxWorks or one you have implemented yourself, the simulator includes support for virtual disks. A virtual disk converts all read and write accesses to read and write accesses to a file on the host file system. However, to an application running in a VxWorks image, accessing the virtual disk looks no different than any other disk in a VxWorks I/O system.

Note: Virtual disk support replaces the UNIX disk driver library (unixDrv) included in earlier versions of the simulator.

By default, the simulator includes support for virtual disks. The relevant configuration component is **INCLUDE_VIRTUAL_DISK**. To initialize the virtual disk system, call virtualDiskInit( ). After control returns from a successful call to virtualDiskInit( ), you can create a virtual disk instance by calling virtualDiskCreate( ):

```
BLK_DEV * vitualDiskCreate
    (
    char * hostFile, /* name of the host file to use */
    int bytesPerBlk, /* number of bytes per block */
    int blksPerTrack, /* number of blocks per track */
    int nBlocks /* number of blocks on this device */
    )
```

Although a successful call to virtualDiskCreate( ) creates a disk instance, there is not yet any name or file system associated with the instance. To create this association, you must call a file system device initialization function. Consider the following code fragment:

```
BLK_DEV * vdBlkDev;
vdBlkDev = virtualDiskCreate (hostFile, 512, 32, 32*200);
fsmNameInstall("/Q:0", "/Q");
xbd = xbdBlkDevCreateSync(vdBlkDev, "/Q");
status = dosFsVolFormat("/Q", DOS_OPT_QUIET | DOS_OPT_BLANK, NULL);
```

This code creates /Q, a 3 MB DOS disk with 512-byte blocks, and 32 tracks. In support of this virtual disk, the virtualDiskCreate( ) call creates the file **c:/tmp/filesys1** (if the file does not already exist). Do not delete this file while the virtual disk is open (to close a virtual disk, call the virtualDiskClose( ) function). To check whether this code has successfully created a virtual disk, you can call devs( ), which returns the following:

```
drv name
    0 /null
    1 /ttyS0
    5 host:
    6 /vio
    3 C:
```

Watchpoint Support

VxSim supports watchpoints—hardware breakpoints (starting with layer version simulator-1.0.7.3). There are four debug registers available.

Note: As with Linux, VxSim watchpoint support requires ptrace( ). It may be necessary to enable ptrace( ) support on the host before VxSim can set debug registers (for example on Ubuntu distributions set **proc/sys/kernel/yama/ptrace_scope** to 1).

WNDRVR

8. Migrating Applications to a Hardware-Based System

Kernel and RTP applications developed using the simulator are easily transferred to target hardware systems.

However, because the simulator environment is not a suitable basis for developing hardware device drivers, more work may be required once your application is transferred to the target system.

To migrate your application, change your project build specifications to reflect the new hardware-based system. This involves recompiling your code using the appropriate CPU type for your target hardware.

WNDRVR

11. ACCESSING HOST RESOURCES

1. Host Functions and Code Access

The VxSim provides support to access the underlying host OS functions from a VxWorks application and to call host code stored in a dynamic-link library (DLL).

That is, you can write a generic DLL (on any host) to control a hardware device connected to the host. The DLL can then be loaded by, and accessed through, the simulator.

For information on the available host access functions, see the reference entry for **vxsimHostArchLib**. **vxsimHostArchLib** also provides a host library (**vxsimapi**) for simulator host application development. For more information, see the reference entry for **vxsimapi**.

2. Accessing Host OS Functions

The vxsimHostProcAddrGet( ) function allows you to retrieve the address of a host function.

For example, the following code retrieves the address of the underlying host OS malloc( ) function:

```
/* Get underlying host OS malloc() address */
pHostMalloc = vxsimHostProcAddrGet ("malloc");

/* Allocate a buffer on host side of VxWorks Simulator */
lvl = intLock (); /* lock interrupts */
pHostBuf = (*pHostMalloc) (0x1000);
intUnlock (lvl); /* unlock interrupts */
```

You should observe the following guidelines when making a call to host code:

- When you call a host function from VxWorks code, you must always lock interrupts before calling the function. Failure to do so can result in unexpected simulator behavior.
- When you call a host function and the function is system blocking, the functions will not only block the VxWorks task from which it was called, but will also block the entire simulator.

  To avoid blocking on a Windows simulator, create a specific thread that is responsible for calling the potentially blocking host code and set up a simple communication mechanism between VxWorks and the thread you have created. For an example of this, see Controlling a Host Serial Device on page 91.

  The method described for Windows simulators cannot be used on Linux simulators because these simulators do not support multithreading. On these hosts, if the blocking system call is a device access, the solution is to configure the host device to generate an interrupt when data becomes available. For more information on using this method, see Host Application Interface (vxsimapi) on page 87.

3. Accessing Host File System Resources

The default VxWorks simulator file system is pass-through (**passFs**). It is included by default in the VIP through the component **INCLUDE_PASSFS** which mounts the file system on startup.

PassFs is a file-oriented device driver which provides easy access to the host file system.

WNDRVR

From the VxWorks Kernel shell, you can find the passFs device name by executing the devs( ) function.

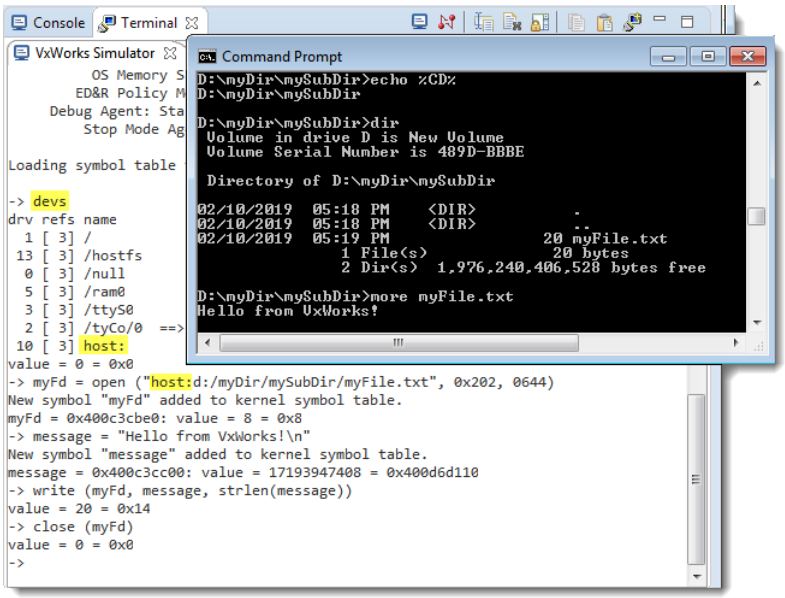
Windows Host

Driver letters are not recognized on a windows based VxSim. You need to add the drive letter after **host:** (or **passFs-device-name:**). The VxWorks syntax to access a host file system is: <passFs-device-name>:<driver-letter>:<absolute-path>/<file-name>

On a Windows host, the default passFs device name is **host**.

- To open the **D:\myDir\mySubDir\myFile.txt** file on a Windows host machine, from the kernel shell:

```
myFd = open ("host:d:/myDir/mySubDir/myFile.txt", 0x202, 0644)
```



Linux Host

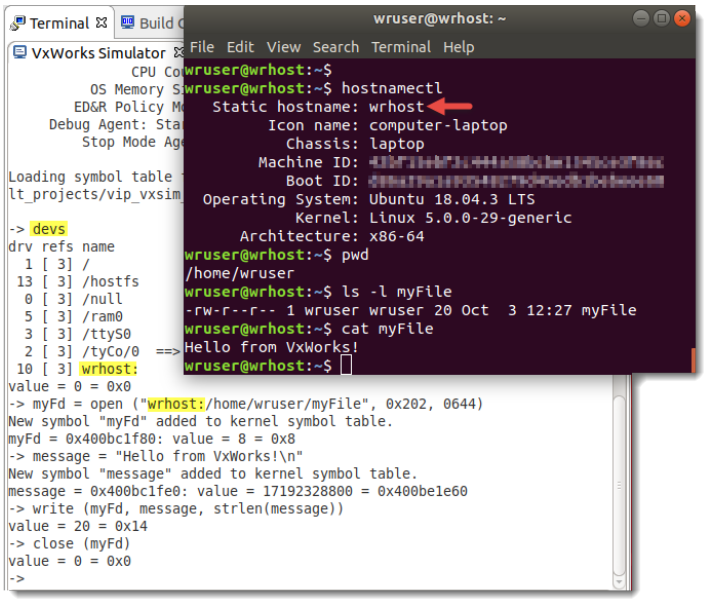
On a UNIX VxSim, the VxWorks syntax to access a host file system is: <passFs-device-name>:<absolute-path-to-file>/<file-name>

On a Linux host, the default passFs device name is the hostname of the machine and you can view the current hostname by executing the hostnamectl command.

> Note: hostnamectl is a linux commnad. devs( ) is a VxWorks function.
>
> To ensure VxWorks recognizes the hostname execute the hostnamectl command in linux, then execute the devs( ) function in VxWorks shell.

- To open the **home/wruser/myFile** file on a Linux host machine named **wrhost**, from the kernel shell:

```
myFd = open ("wrhost:/home/wruser/myFile", 0x202, 0644)
```



4. Loading a Host-Based Application

The vxsimHostDllLoad( ) function provides the ability to load a DLL in the simulator process.

The exported symbols of the DLL can then be retrieved using the vxsimHostProcAddrGet( ) function described in Accessing Host OS Functions on page 85.

5. Host Application Interface (vxsimapi)

The **vxsimapi** library provides the ability to extend simulator capabilities with native OS code to perform operations that cannot be done directly using VxWorks code.

For example, this facility can be used to add code to control peripherals connected to the host machine, to add code for graphic applications, or to add any functionality that requires host-specific code. For more information, see the reference entry for **vxsimapi**.

Defining User Exit Hooks

Applications often need to perform specific actions on exit. This includes items such as releasing resources, re-initializing peripherals, and other cleanup operations. The simulator exit hook facility provides the vxsimExitHookAdd( ) function. This function gives you the ability to specify a function to perform any necessary cleanup when the simulator exits or reboots.

Configuring a Host Device to Generate Interrupts (Linux Only)

You can put the host file descriptors in asynchronous mode, such that VxWorks sends a **SIGPOLL** signal is sent to the simulator when data becomes available. If a VxWorks task reads from a host device, the task normally requires a blocking read. Because Linux simulators are mono-threaded, this action stops the simulator process entirely until data is ready. As an alternative, you can open the file in non-blocking mode and then put the device into asynchronous mode. This causes a **SIGPOLL** signal to be sent whenever data becomes available. In this case, an input ISR reads the data, puts it in a buffer, and unblocks the waiting task.

WNDRVR

To install an ISR that runs whenever data is ready on some underlying host device, you must first open the host device in non-blocking mode. Then, put the file descriptor in asynchronous mode using the vxsimFdIntEnable( ) function. This ensures that the host will send a SIGPOLL signal when data is available. On the target side, an interrupt service function (ISR) is connected using vxbIntConnect( ).

For a serial port in the host code, for example:

```
DLL_EXPORT UINT32 ttySioOpen (...)
    {
    ...
    serial.fd = open (SERIAL_DEV(serial.portNum),
        O_RDWR | O_NONBLOCK | O_NOCTTY);
    ...
    return (serial.fd);
    }
DLL_EXPORT STATUS ttySioIntSet (...)
    {
    ...
    vxsimFdIntEnable (serial.fd);
    ...
    }
```

For more information, see *installDir* **/vxworks-7/host/src/vxsim/ttySio.c**, which provides functions to open a host device and configure it (the ttySioOpen( ) function), and to set interrupt mode (the ttySioModeSet( ) function).

In the target code, the **vxbVxsimHostSio.c** driver (in *installDir* **/vxworks-7/pkgs_v2/os/arch/simulator-***versionNum* **/kernel/src/drv**) will call host routines through vxsimHostProcCall( ) to open a device, configure it, and connect the interrupt with vxbIntConnect( ).

Simulating Interrupts From a User Application (Windows Only) on page 88 The vxsimIntRaise( ) function provides a host side application with the ability to notify VxWorks of a given event, allowing VxWorks to take the appropriate action.

5.1. Simulating Interrupts From a User Application (Windows Only)

The vxsimIntRaise( ) function provides a host side application with the ability to notify VxWorks of a given event, allowing VxWorks to take the appropriate action.

For example, if you have an application collecting data from a device, you can raise an interrupt to VxWorks when data has been read from the device. On the target side of the application, an ISR can be connected to the interrupt vector using vxbIntConnect( ). Now, each time vxsimIntRaise( ) is called, the ISR is called to handle the read data. (See the host code in *installDir* **/vxworks-7/host/src/vxsim/serial/commSio.c**, and the corresponding target driver code in *installDir* **/vxworks-7/pkgs_v2/os/arch/simulator-***versionNum* **/kernel/src/drv/vxbVxsimHostSio.c.**)

When an interrupt needs to be acknowledged, the vxsimIntAckRtnAdd( ) function can be used to connect an acknowledgement function for a given interrupt vector. This function is called immediately after the interrupt handling. A range of interrupt vectors are available on Windows simulators.

This range is defined in the **config.h** file of the **simpc_windows** BSP:

USER_INT_RANGE_BASE

User interrupts range base.

USER_INT_RANGE_END

User interrupts range end.

The functions vxsimIntToMsg( ) and vxsimMsgToInt( ) allow you to convert an interrupt vector number to a Windows message number, and conversely, allow you to convert a message number to a vector number.

WNDRVR

The vxsimWindowsHandleGet( ) function can be used with the simulator to get a windows handle for sending messages.

For more information on Windows simulator interrupt assignments, see Interrupts on page 73.

Parent topic: Host Application Interface (vxsimapi) on page 87

6. Running Tcl on the VxWorks Simulator

You can load a standard Tcl DLL on the simulator and start a Tcl interpreter.

```
#include "vxWorks.h"
#include "vxsimHostLib.h"

#if (CPU==SIMNT)
#define TCL_DLL "tcl84.dll"     /* Windows Tcl Dll name */
#else
#define TCL_DLL "libtcl8.4.so"  /* UNIX Tcl Dll name */
#endif

BOOL tclLoaded = FALSE;

STATUS tclStart (void)
    {
    /* Function pointers for Tcl Dll routines */

    FUNCPTR    pTcl_CreateInterp;
    FUNCPTR    pTcl_Init;
    FUNCPTR    pTcl_Eval;
    FUNCPTR    pTcl_GetStringResult;
    FUNCPTR    pTcl_DeleteInterp;

    char       tclCommand[400];     /* buffer for Tcl command */
    int        evalResult;          /* Tcl command evaluation result */
    int        lvl;                 /* interrupt lock level */
    void *     pInterp;             /* Tcl interpreter Id */

    /* load Tcl Dll */

    if (tclLoaded == FALSE)
        {
        if (vxsimHostDllLoad (TCL_DLL) != OK)
            {
            printf ("Error: Failed to load %s\n", TCL_DLL);
            return (ERROR);
            }

        tclLoaded = TRUE;
        }

    /* retrieve some Tcl routine address from the loaded Dll */

    pTcl_CreateInterp    = vxsimHostProcAddrGet ("Tcl_CreateInterp");
    pTcl_Init            = vxsimHostProcAddrGet ("Tcl_Init");
    pTcl_Eval            = vxsimHostProcAddrGet ("Tcl_Eval");
```


WNDRVR

```
    pTcl_GetStringResult = vxsimHostProcAddrGet ("Tcl_GetStringResult");
    pTcl_DeleteInterp    = vxsimHostProcAddrGet ("Tcl_DeleteInterp");
    /* Create and Initialize Tcl interpreter */

    lvl = intLock ();                           /* lock interrupts */
    pInterp = (void *)(*pTcl_CreateInterp) ();  /* Create interpreter */
    (*pTcl_Init) (pInterp);                     /* Initialize interpreter */
    intUnlock (lvl);                            /* unlock interrupts */

    printf ("Tcl Ready (Type CTRL+D to exit interpreter)\n\ntcl> ");

    while (gets (tclCommand) != NULL)
    {
    lvl = intLock ();     /* lock interrupts */

    evalResult = (*pTcl_Eval) (pInterp, tclCommand);

    if (evalResult != 0)
        {
        printf ("Tcl Error: ");
        }

    if (strlen ((*pTcl_GetStringResult)(pInterp)) != 0)
        printf ("%s\n", (*pTcl_GetStringResult)(pInterp));

    intUnlock (lvl);      /* unlock interrupts */

    printf ("tcl> ");
    }

    /* Delete Tcl interpreter */

    lvl = intLock ();    /* lock interrupts */
    (*pTcl_DeleteInterp) (pInterp);
    intUnlock (lvl);     /* unlock interrupts */

    return (OK);
    }
```

Running the Code

The sample code can be executed directly from the VxWorks kernel shell or using the host shell. A sample host shell session is as follows:

```
-> ld < tclInterp.o
-> tclStart
Loading libtcl8.4.so ... succeeded.
Tcl Ready (Type CTRL+D to exit interpreter)
tcl> glob *
tclInterp.c tclInterp.o hello.tcl
tcl> source hello.tcl

Hello !
tcl> ^Dvalue = 0 = 0x0
->
```

7. Controlling a Host Serial Device

Controlling a host serial device is a complex application.

For an example of this application type, see the reference entry for **commSio** (Windows simulators) or **ttySio** (Linux simulators). The examples provided for **commSio** and **ttySio** exercise most of the available features.

12. VXWORKS SIMULATOR EXCEPTIONS

1. Simulator Exceptions for Windows Reference

The simulator generates a set of exceptions for Windows.

The simulator name for an exception has a corresponding POSIX name.

The following table lists exceptions generated for the Windows operating system.

VxWorks Software Signal	POSIX Software Signal	Simulator Exception Name	Description
SIGFPE	**SIGFPE**	**EXC_INT_DIVIDE_BY_ZERO**	Divide by zero
SIGEMT	**SIGTRAP**	**EXC_SINGLE_STEP**	Single step
SIGSEGV	**SIGBUS**	**EXC_DATATYPE_MISALIGNMENT**	Datatype misalignment
SIGEMT	**SIGTRAP**	**EXC_BREAKPOINT**	Breakpoint
SIGSEGV	**SIGSEGV**	**EXC_IN_PAGE_ERROR**	In-page error
SIGILL	**SIGILL**	**EXC_ILLEGAL_INSTRUCTION**	Illegal instruction
SIGILL	**SIGILL**	**EXC_INVALID_DISPOSITION**	Invalid disposition
SIGBUS	**SIGSEGV**	**EXC_ARRAY_BOUNDS_EXCEEDED**	Array bounds exceeded
SIGILL	**SIGFPE**	**EXC_FLT_DENORMAL_OPERAND**	Floating point denormal operand
SIGFPE	**SIGFPE**	**EXC_FLT_DIVIDE_BY_ZERO**	Floating point divide by zero
SIGFPE	**SIGFPE**	**EXC_FLT_INEXACT_RESULT**	Floating point inexact result
SIGFPE	**SIGFPE**	**EXC_FLT_INVALID_OPERATION**	Floating point invalid operation
SIGFPE	**SIGFPE**	**EXC_FLT_OVERFLOW**	Floating point overflow
SIGSEGV	**SIGSEGV**	**EXC_ACCESS_VIOLATION**	Access violation
SIGBUS	**SIGBUS**	**EXC_FLT_STACK_CHECK**	Floating point stack check
SIGFPE	**SIGFPE**	**EXC_FLT_UNDERFLOW**	Floating point underflow
SIGILL	**SIGILL**	**EXC_PRIV_INSTRUCTION**	Private instruction

WNDRVR

VxWorks Software Signal	POSIX Software Signal	Simulator Exception Name	Description
SIGBUS	**SIGBUS**	**EXC_STACK_OVERFLOW**	Stack overflow
SIGILL	**SIGILL**	**EXC_UNKNOWN**	Unknown exception

2. Simulator Exceptions for Linux

The simulator generates a set of exceptions for Linux.

The simulator name for an exception has a corresponding POSIX name. The following table lists exceptions generated for Linux operating systems.

VxWorks Software Signal	POSIX Software Signal	Simulator Exception Name	Description
SIGSEGV	**SIGSEGV**	**IV_SEGV**	Segmentation violation
SIGBUS	**SIGBUS**	**IV_BUS**	BUS error
SIGILL	**SIGILL**	**IV_ILL**	Illegal instruction
SIGFPE	**SIGFPE**	**IV_FPE**	Floating point exception
SIGTRAP	**SIGTRAP**	**IV_TRAP**	Trap exception

WNDRVR